# CENG7880
# Trustworthy and Responsible AI

Instructor: Sinan Kalkan

(https://ceng.metu.edu.tr/~skalkan)

For course logistics and materials:
https://metu-trai.github.io



MIDDLE EAST TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

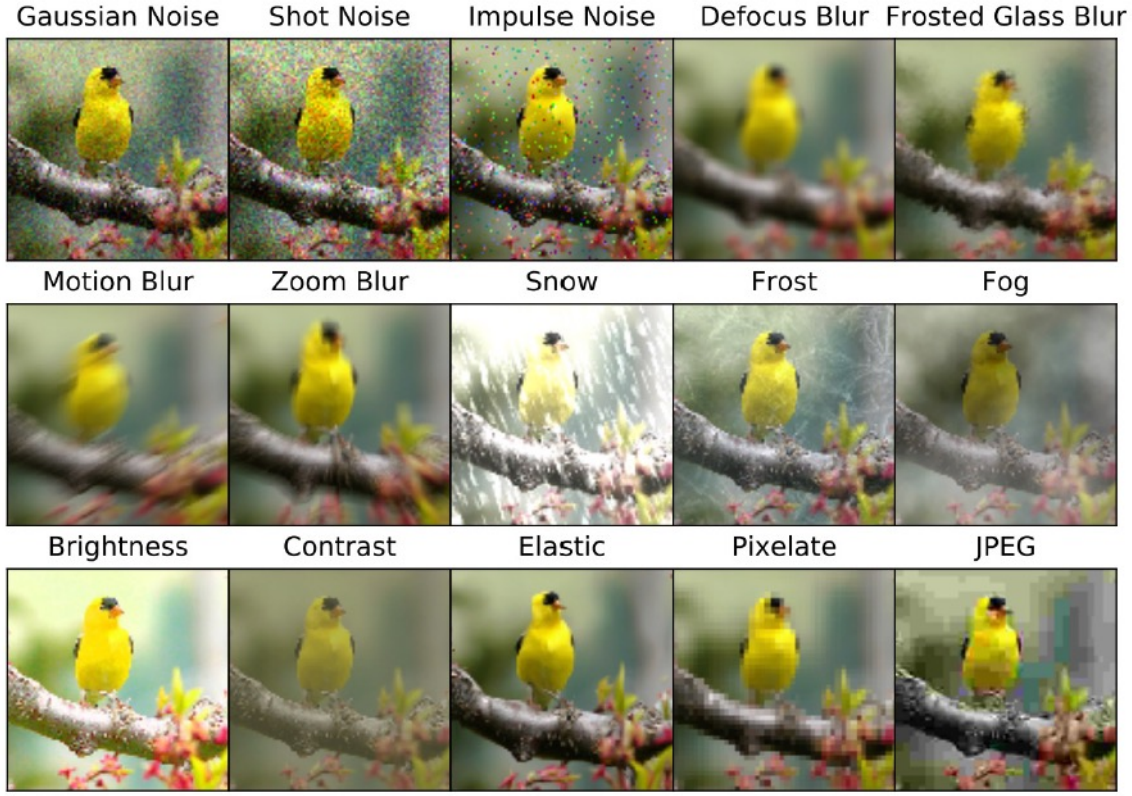


Previously on CENG7880!

# Robustness: Agenda

## Robustness to Distribution Shifts

- Images
  - Noise, color changes, light changes, day/night, summer/winter, …
- Audio
  - Noise, background, gain level, …
- Text
  - Noise, synonyms, alternative phrasing, ..



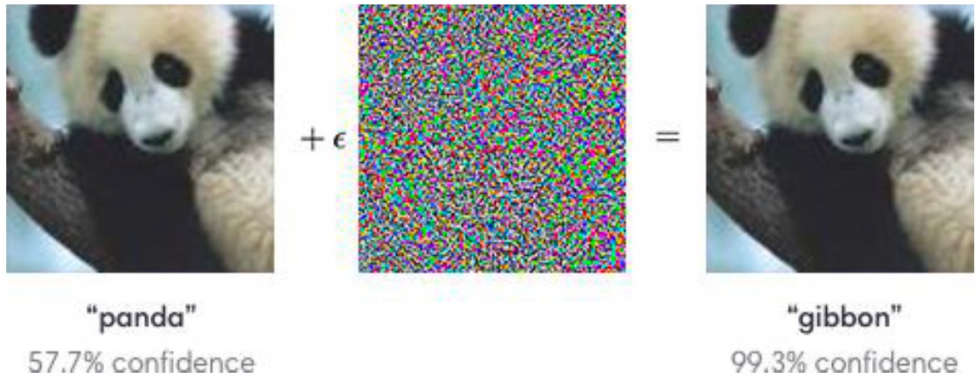

Hendrycks & Dietterich, Benchmarking Neural Network Robustness to Common Corruptions and Perturbations

Previously on CENG7880!

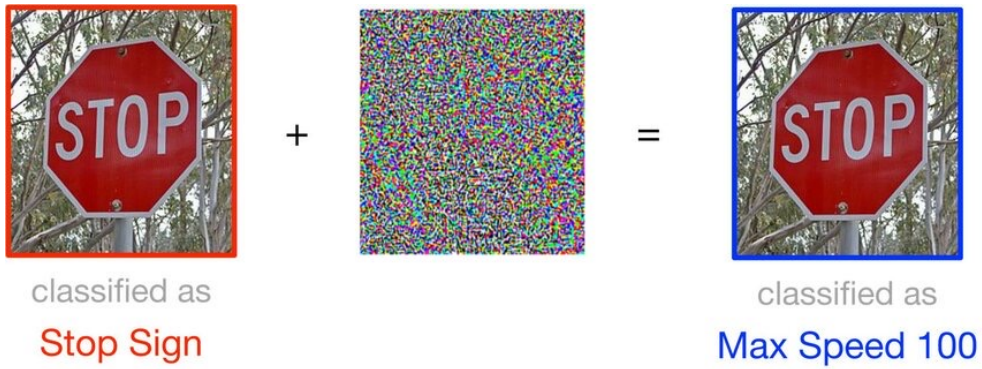
# Robustness: Agenda
## Adversarial Robustness

- Types of adversarial attacks

- Adversarial training




Szegedy et al., Intriguing Properties of Neural Networks, 2014

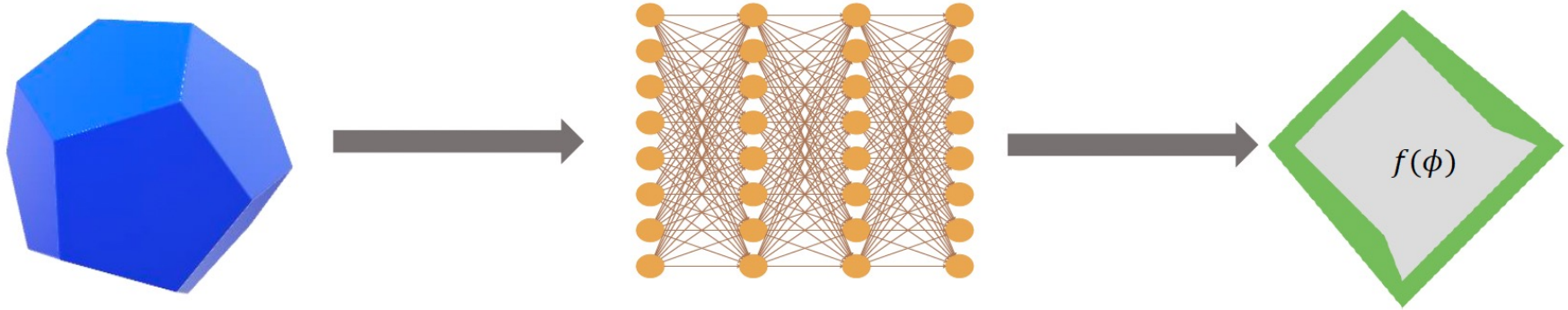



https://www.altacognita.com/robust-attribution/

Previously on CENG7880!

# Robustness: Agenda
## Certifying/Verifying Robustness

### Neural network certification: problem statement




Figs: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Previously on CENG7880!

# Robustness: Agenda

## Calibrated Predictions & Uncertainty

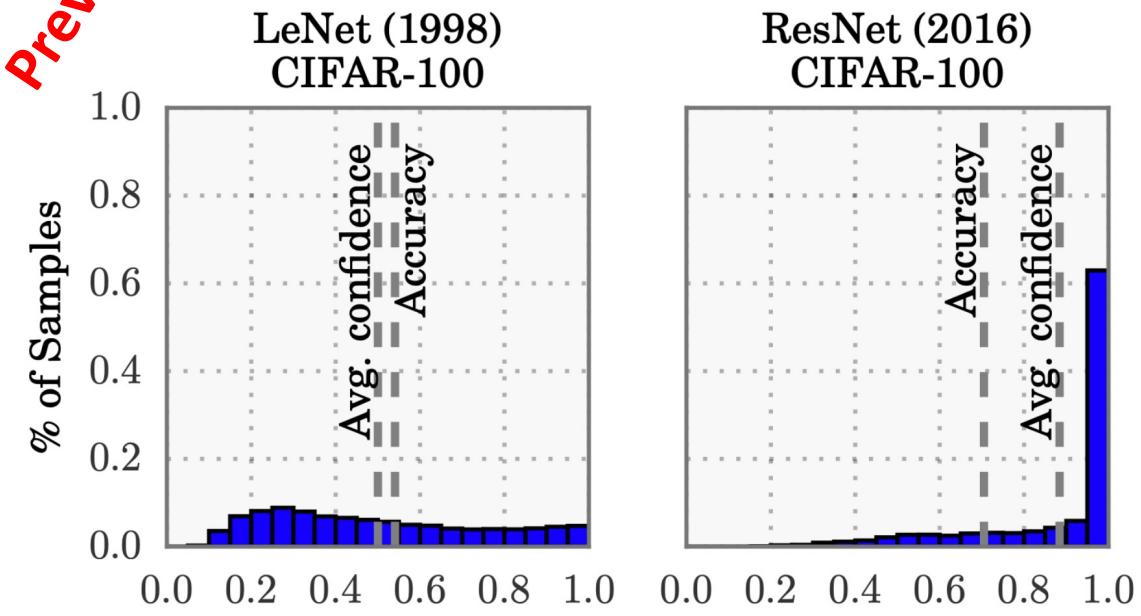



Fig: Uni of Pennsylvania, Trustworthy ML - CIS 7000

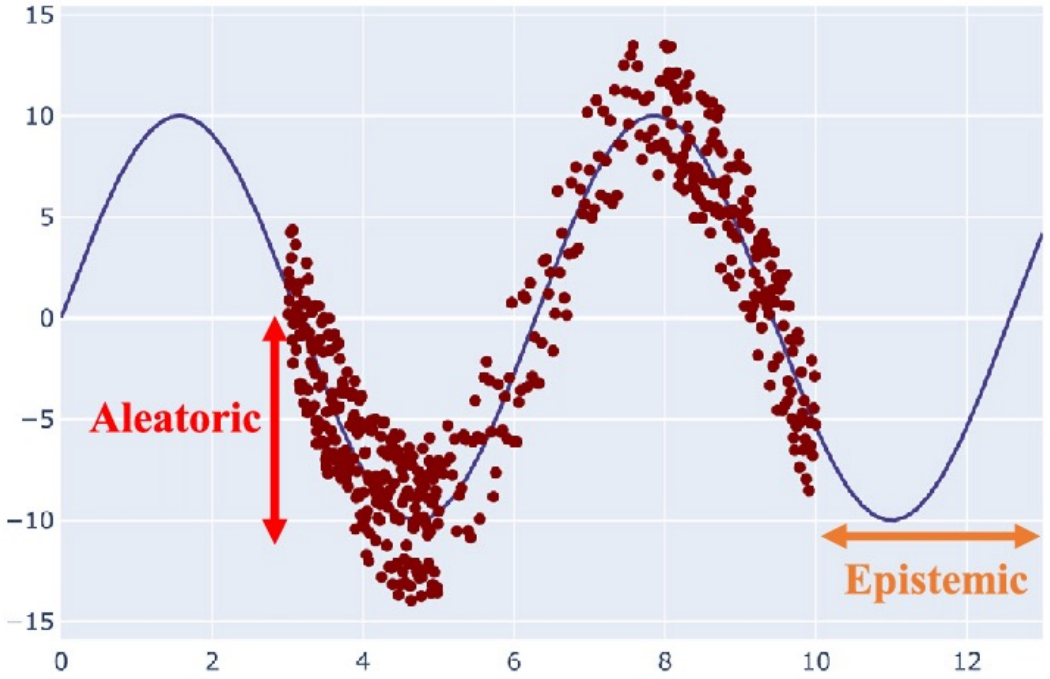



Fig: Abdar et al., A review of uncertainty quantification in deep learning: Techniques, applications and challenges, 2021

# Distribution Shift

Previously on CENG7880!

- **Distribution shift:** Training and test distributions differ
  - Training set consists of samples $(x_1, y_1), \ldots, (x_n, y_n) \sim P$
  - Test set consists of samples $(x'_1, y'_1), \ldots, (x'_m, y'_m) \sim Q$

- **Supervised learning under distribution shift**
  - Given training dataset $Z \subseteq \mathcal{X} \times \mathcal{Y}$ consisting of i.i.d. samples $(x, y) \sim P$
  - Goal is to minimize loss $\mathbb{E}_Q[\ell(\theta; x, y)] \not\approx |Z|^{-1} \sum_{(x,y)\in Z} \ell(\theta; x, y)$
  - Computing $\hat{\theta} = \min_\theta |Z|^{-1} \sum_{(x,y)\in Z} \ell(\theta; x, y)$ may not work



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Label Shift Assumption

Previously on CENG7880!

- Let $p$ and $q$ be the density functions for $P$ and $Q$, respectively

- **Label Shift Assumption:** $p(y) \neq q(y)$
  - But $p(x \mid y) = q(x \mid y)$
  - **Intuition:** The rates of labels changes, but the kinds of labels don't

# Covariate Shift Assumption

Previously on CENG7880!

- Let $p$ and $q$ be the density functions for $P$ and $Q$, respectively

- **Covariate Shift Assumption:** $p(x) \neq q(x)$
  - But $p(y \mid x) = q(y \mid x)$
  - **Intuition:** The label computation does not change, but the inputs can change



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Importance Weighting

Previously on CENG7880!

- Given distributions $P$ and $Q$, the **importance weight (function)** is

$$w(x, y) = \frac{q(x, y)}{p(x, y)}$$

- **Key property (by definition):**

$$\mathbb{E}_Q[\ell(\theta; x, y)] = \mathbb{E}_P[\ell(\theta; x, y) \cdot w(x, y)]$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Supervised Learning with Label Shift

Previously on CENG7880!

- **Input:** Training dataset $Z$, unlabeled test dataset $X$
- **Step 1:** Train $f$ on $Z$
- **Step 2:** Estimate using the dataset:
  - $C_{ij} = \mathbb{P}_P[f(x) = i, y = j] \approx |Z|^{-1} \sum_{(x,y)\in Z} 1(f(x) = i \wedge y = j)$
  - $q_i = \mathbb{P}_Q[f(x) = i] \approx |X|^{-1} \sum_{x\in X} 1(f(x) = i)$
- **Step 3:** Compute $w = C^{-1}q$
- **Step 4:** Compute $\hat{\theta} = \arg\min_{\theta} \sum_{(x,y)\in Z} \ell(\theta; x, y) \cdot w(y)$

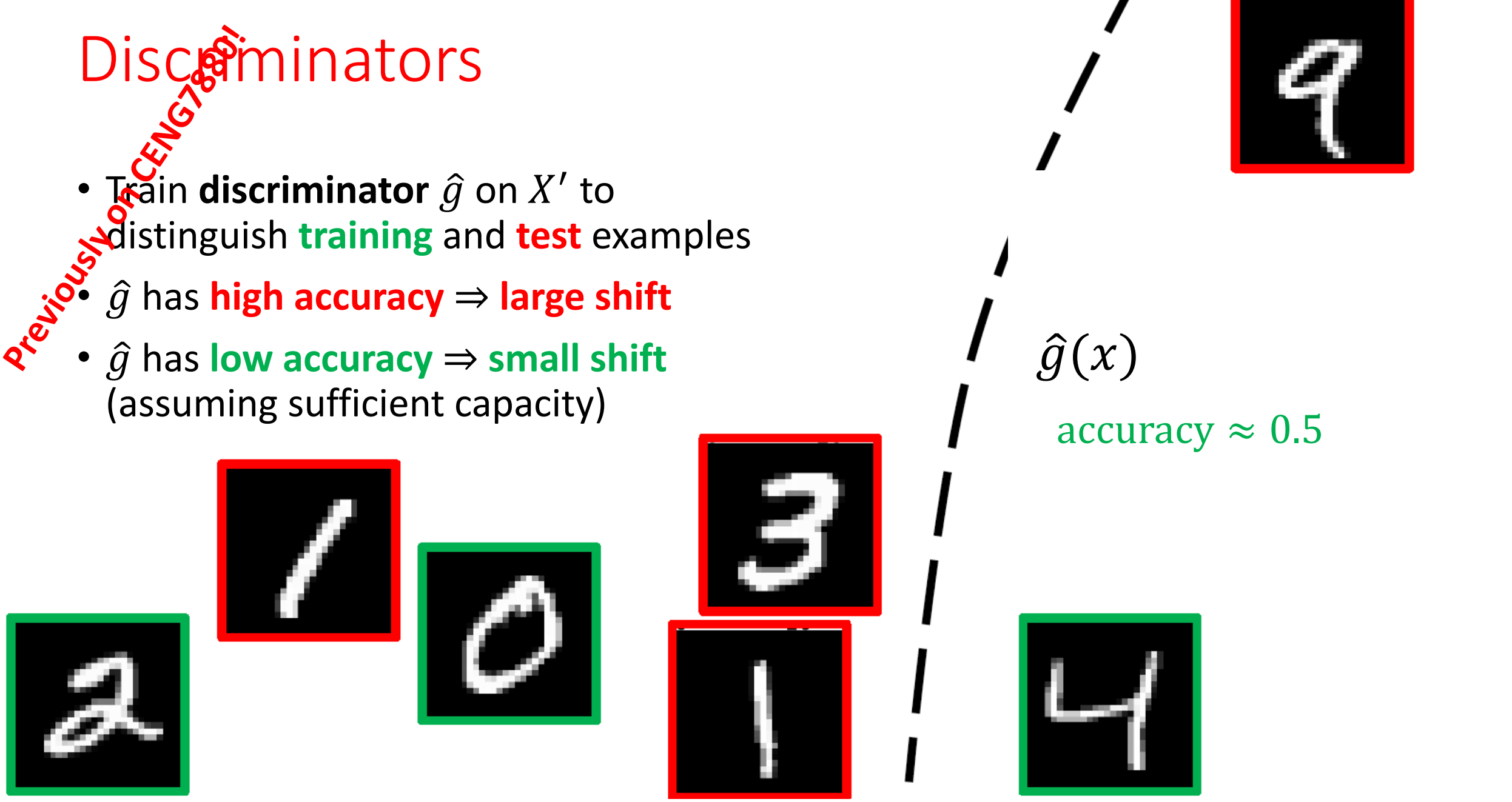


Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Discriminators

Previously on CENG7880!

- Train **discriminator** $\hat{g}$ on $X'$ to distinguish **training** and **test** examples
- $\hat{g}$ has **high accuracy** ⇒ **large shift**
- $\hat{g}$ has **low accuracy** ⇒ **small shift** (assuming sufficient capacity)






Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Supervised Learning with Covariate Shift

Previously on CENG7880!

- **Input:** Training dataset $Z$, unlabeled test dataset $X$
- **Step 1:** Construct $X' = \{(x, 0) \mid (x, y) \in Z\} \cup \{(x, 1) \mid x \in X\}$ and train $\hat{g}$ on $X'$ to predict $b$ given $x$
- **Step 2:** Compute $w(x) = \frac{1}{\hat{g}(b=0 \mid x)} - 1$
- **Step 3:** Compute $\hat{\theta} = \arg\min_{\theta} \sum_{(x,y)\in Z} \ell(\theta; x, y) \cdot w(x)$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Covariate Shift Tutorial

https://colab.research.google.com/drive/1nQmbDksZWaSNbnyxCwErcSVcVioEFk-I?usp=sharing

by Melih Darcan

# Agenda

- Robustness
  - Distribution shifts (this week)
  - Adversarial robustness (this week)
  - Certifying robustness (next week)
  - Calibrated predictions & uncertainty (next next week)

# Administrative Notes

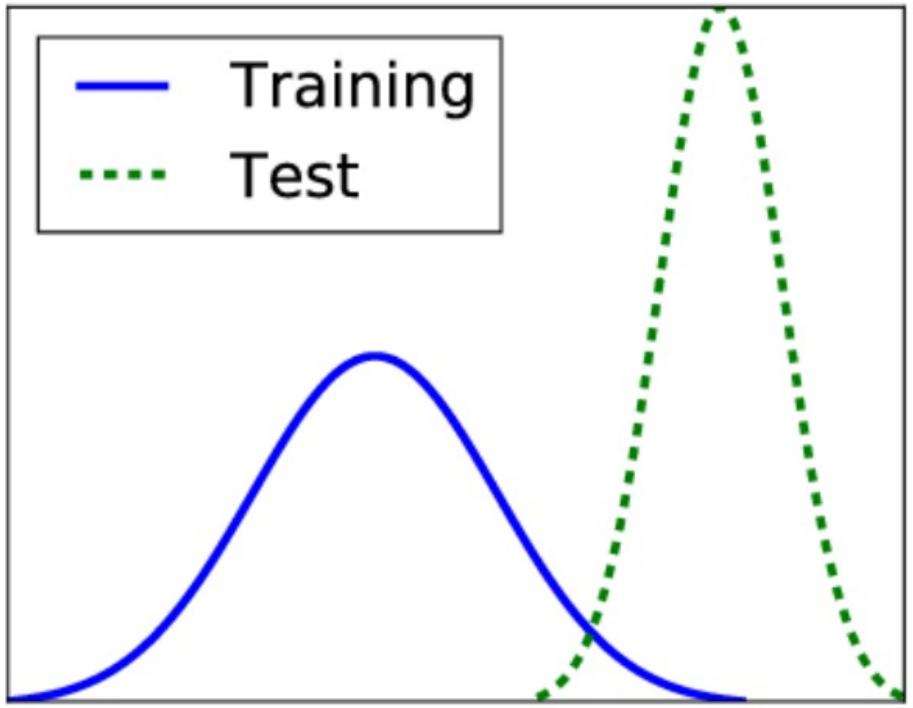
- Selecting papers
  - ~~Form: https://forms.gle/Z8SFya9hQtdzP7jG6~~
  - ~~Deadline for choosing papers: 16 October.~~

# Evaluation Bounds for Distributional Robustness

and Distributionally Robust Optimization

# Support of Shifted Data

- **Assumption:** Support of $Q$ is not contained in support of $P$
- However, this is **necessary** since we do not know anything about data outside of the support of $P$
- Need additional assumptions to do better
  - Focus on covariate shift




Image; Glauner et al., 2018



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Learning vs. Evaluation

- For this part, we will focus on **model evaluation**
  - **Learning:** Optimize $\mathbb{E}_Q[\ell(\theta; x, y)]$
  - **Evaluation:** Estimate $\mathbb{E}_Q[\ell(\theta; x, y)]$

- We will see why learning is harder later



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Integral Probability Metrics

- The **total variation distance** is

$$\mathrm{TV}(P,Q) = \int_{\mathcal{X}\times\mathcal{Y}} |q(x,y) - p(x,y)| \cdot dx \cdot dy$$

Sensitive to the mis-alignment between the two distributions. No notion of cost between two points.

- The **Wasserstein distance** is

$$W(P,Q) = \sup_{f:K_f \leq 1} \int_{\mathcal{X}\times\mathcal{Y}} f(x,y) \cdot (q(x,y) - p(x,y)) \cdot dx \cdot dy$$

$f(x,y)$: A smooth-function (Lipschitz constant less than 1) that measures how far apart the two distributions are. We are trying to find $f(x,y)$ that best separates the distributions.



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds

- Note that

$$\mathbb{E}_Q[\ell(\theta; x, y)] = \int_{X \times Y} \ell(\theta; x, y) \cdot q(x, y) \cdot dx \cdot dy$$

$$= \int_{X \times Y} \ell(\theta; x, y) \cdot (p(x, y) + q(x, y) - p(x, y)) \cdot dx \cdot dy$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds

- Note that

$$\mathbb{E}_Q[\ell(\theta; x, y)] = \int_{X \times Y} \ell(\theta; x, y) \cdot q(x, y) \cdot dx \cdot dy$$

$$= \int_{X \times Y} \ell(\theta; x, y) \cdot (p(x, y) + q(x, y) - p(x, y)) \cdot dx \cdot dy$$

$$= \int_{X \times Y} \ell(\theta; x, y) \cdot p(x, y) \cdot dx \cdot dy$$

$$+ \int_{X \times Y} \ell(\theta; x, y) \cdot (q(x, y) - p(x, y)) \cdot dx \cdot dy$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds

- Note that

$$\begin{aligned}
\mathbb{E}_Q[\ell(\theta; x, y)] &= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot q(x, y) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (p(x, y) + q(x, y) - p(x, y)) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot p(x, y) \cdot dx \cdot dy \\
&\quad + \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (q(x, y) - p(x, y)) \cdot dx \cdot dy \\
&= \mathbb{E}_P[\ell(\theta; x, y)] + \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (q(x, y) - p(x, y)) \cdot dx \cdot dy
\end{aligned}$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds

- Note that

$$
\begin{aligned}
\mathbb{E}_Q[\ell(\theta;x,y)] &= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta;x,y)\cdot q(x,y)\cdot dx\cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta;x,y)\cdot (p(x,y)+q(x,y)-p(x,y))\cdot dx\cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta;x,y)\cdot p(x,y)\cdot dx\cdot dy \\
&\quad + \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta;x,y)\cdot (q(x,y)-p(x,y))\cdot dx\cdot dy \\
&= \mathbb{E}_P[\ell(\theta;x,y)] + \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta;x,y)\cdot (q(x,y)-p(x,y))\cdot dx\cdot dy \\
&\le \mathbb{E}_P[\ell(\theta;x,y)] + \ell_{\max}\cdot \int_{\mathcal{X}\times\mathcal{Y}} |q(x,y)-p(x,y)|\cdot dx\cdot dy \\
&= \mathbb{E}_P[\ell(\theta;x,y)] + \ell_{\max}\cdot \mathrm{TV}(P,Q)
\end{aligned}
$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds

- Note that

$$
\begin{aligned}
\mathbb{E}_Q[\ell(\theta; x, y)] &= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot q(x, y) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (p(x, y) + q(x, y) - p(x, y)) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot p(x, y) \cdot dx \cdot dy \\
&\quad + \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (q(x, y) - p(x, y)) \cdot dx \cdot dy \\
&= \mathbb{E}_P[\ell(\theta; x, y)] + \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (q(x, y) - p(x, y)) \cdot dx \cdot dy \\
&\leq \mathbb{E}_P[\ell(\theta; x, y)] + K_\ell \cdot W(P, Q)
\end{aligned}
$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds for Covariate Shift

- Note that

$$
\begin{aligned}
&\int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (q(x, y) - p(x, y)) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (q(y \mid x)q(x) - p(y \mid x)p(x)) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot (p(y \mid x)q(x) - p(y \mid x)p(x)) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}\times\mathcal{Y}} \ell(\theta; x, y) \cdot p(y \mid x) \cdot (q(x) - p(x)) \cdot dx \cdot dy \\
&= \int_{\mathcal{X}} \left( \int_{\mathcal{Y}} \ell(\theta; x, y) \cdot p(y \mid x) \cdot dy \right) \cdot (q(x) - p(x)) \cdot dx \\
&= \int_{\mathcal{X}} \tilde{\ell}(\theta; x) \cdot (q(x) - p(x)) \cdot dx \\
&\leq K_{\tilde{\ell}} \cdot W(P(x), Q(x))
\end{aligned}
$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation Bounds for Covariate Shift

- Thus, we have

$$\mathbb{E}_Q[\ell(\theta; x, y)] \leq \mathbb{E}_P[\ell(\theta; x, y)] + K_{\tilde{\ell}} \cdot W(P(x), Q(x))$$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Aside: What About Learning?

- Suppose we want optimize the upper bound:

$$\mathbb{E}_Q[\ell(\theta; x, y)] \leq \mathbb{E}_P[\ell(\theta; x, y)] + K_{\tilde{\ell}} \cdot W(P(x), Q(x))$$

- It is equivalent to optimizing $\mathbb{E}_P[\ell(\theta; x, y)]$, since the penalty is independent of $\theta$

- Recent approaches to use such bounds for learning

Adapted from Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Solutions to Optimizing the Upper Bound

$$\mathbb{E}_Q[\ell(\theta; x, y)] \leq \mathbb{E}_P[\ell(\theta; x, y)] + K_{\tilde{\ell}} \cdot W(P(x), Q(x))$$

1. Distributionally Robust Optimization: Assume a family/ball of plausible Q and try to minimize data loss for that family of Q as well.

Deterministic Optimization

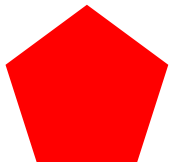
$$\min_{\theta} \ell(x; \theta)$$




Robust Optimization

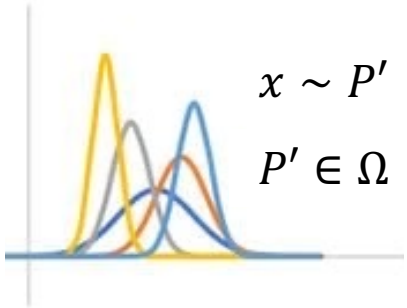
$$\min_{\theta} \max_{x \in X} \ell(x; \theta)$$

$X$: formally characterizable changes




Distributionally Robust Optimization

$$\inf_{\theta} \sup_{P' \in \Omega} \mathbb{E}_{P'}[\ell(x; \theta)]$$



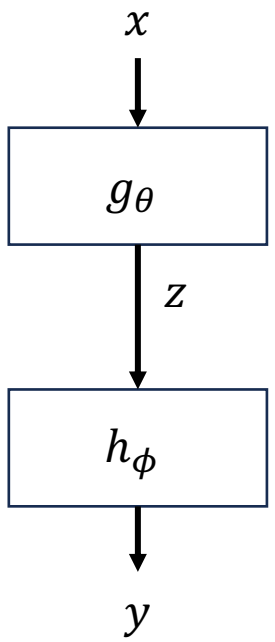
# Solutions to Optimizing the Upper Bound

$$\mathbb{E}_Q[\ell(\theta; x, y)] \leq \mathbb{E}_P[\ell(\theta; x, y)] + K_{\tilde{\ell}} \cdot W(P(x), Q(x))$$

2. Formulate $W()$ in the latent feature space, instead of raw input.

- For a fixed input space $\mathcal{X}$, split the model into a feature extractor $g_\theta: \mathcal{X} \to \mathcal{Z}$ and a classifier/prediction head $h_\phi: \mathcal{Z} \to \mathcal{Y}$.
- The bound is then applied in the latent feature space $\mathcal{Z}$, turning the distance $W\left(g_\theta(x_P), g_\theta(x_Q)\right)$ into a function of the parameter $\theta$:

$$\min_{\theta,\phi} \mathbb{E}_P\left[\ell\left(h_\phi\left(g_\theta(x)\right), y\right)\right] + \lambda \cdot W\left(g_\theta(x_P), g_\theta(x_Q)\right)$$




Methods: Wasserstein Distance Guided Representation Learning, Domain-Adversarial Neural Networks, …

# Solutions to Optimizing the Upper Bound

$$\mathbb{E}_Q[\ell(\theta; x, y)] \leq \mathbb{E}_P[\ell(\theta; x, y)] + K_{\tilde{\ell}} \cdot W(P(x), Q(x))$$

3. Invariant Risk Minimization:

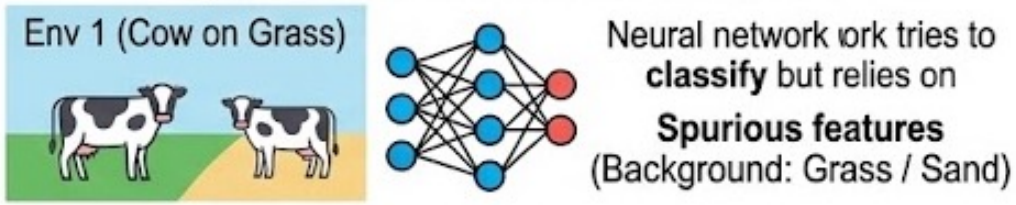
- Learn causal factors/variables/features $z = g_\theta(x)$ such that a linear classifier $h_\phi(z)$ constructed on top of $z$ is optimal across different distributions/environments.

$$\min_{\theta,\phi} \sum_{e \in \mathcal{E}_{tr}} \left(\mathbb{E}_e\left[\ell\left(h_\phi\left(g_\theta(x)\right), y\right)\right] + \lambda \cdot \left\|\nabla_{w|w=1.0}\mathbb{E}_e\left[\ell\left(w \cdot h_\phi\left(g_\theta(x)\right), y\right)\right]\right\|^2\right) + K \cdot W\left(g_\theta\left(x_{\mathcal{E}_{tr}}\right), g_\theta\left(x_Q\right)\right)$$

Minimize prediction loss & its weighted form over different environments/domains

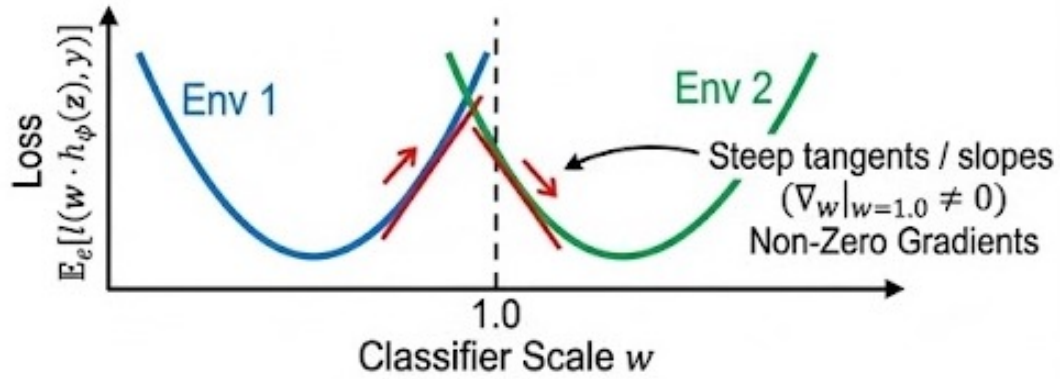
**Left: Non-Invariant** (Using Spurious Features)

**Spurious/Non-Invariant Features**




Neural network ork tries to **classify** but relies on **Spurious features** (Background: Grass / Sand)

Correlation Shifts between domains!

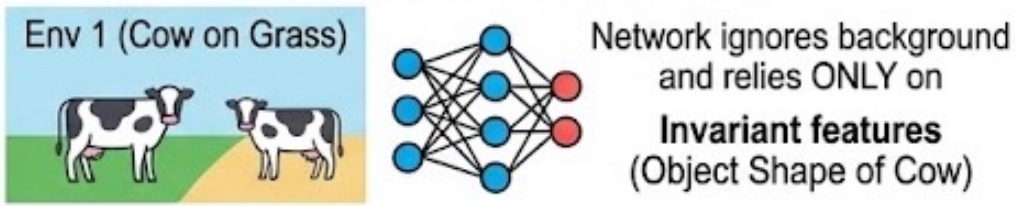



Steep slopes (gradient ≠ 0) → HIGH PENALTY!

(Model penalized for relying on environment-dependent shortcuts)

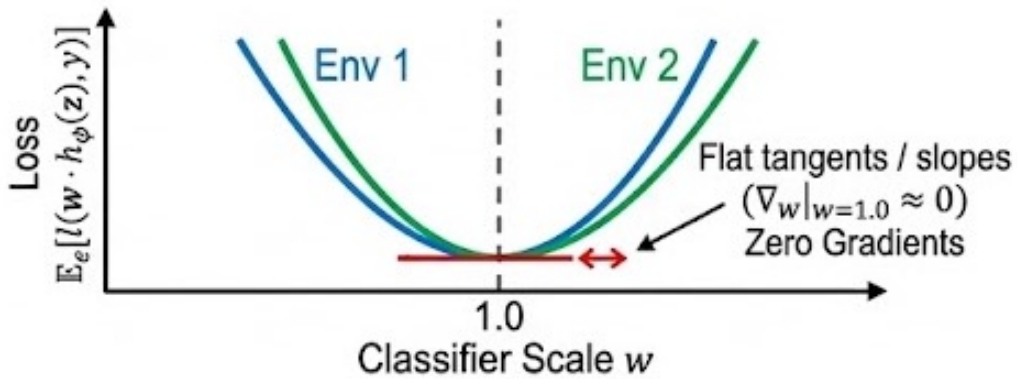
**Right: Invariant** (Using Causal Features)

**Causal/Invariant Features**




Network ignores background and relies ONLY on **Invariant features** (Object Shape of Cow)

Relation is stable across domains!




Flat slopes (gradient ≈ 0) → ZERO PENALTY!

(Predictor is simultaneously optimal, regardless of environment)

**IRM Invariance Penalty:** $\lambda \cdot \left\| \nabla_{\boldsymbol{w}}|_{w=1.0} \, \mathbb{E}_e[l(\boldsymbol{w} \cdot h_\phi(g_\theta(\boldsymbol{x})), y)] \right\|^2$

← Tries to align the natural optimal point of the classifier to exactly w=1.0 across all environments.

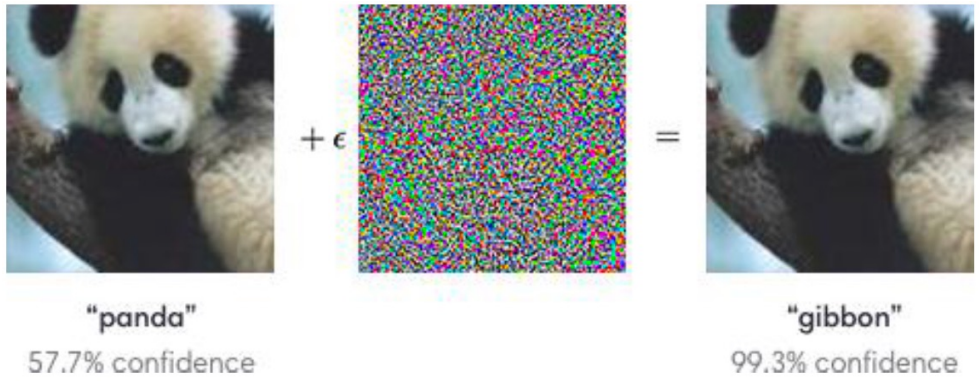


(Prepared with Gemini)

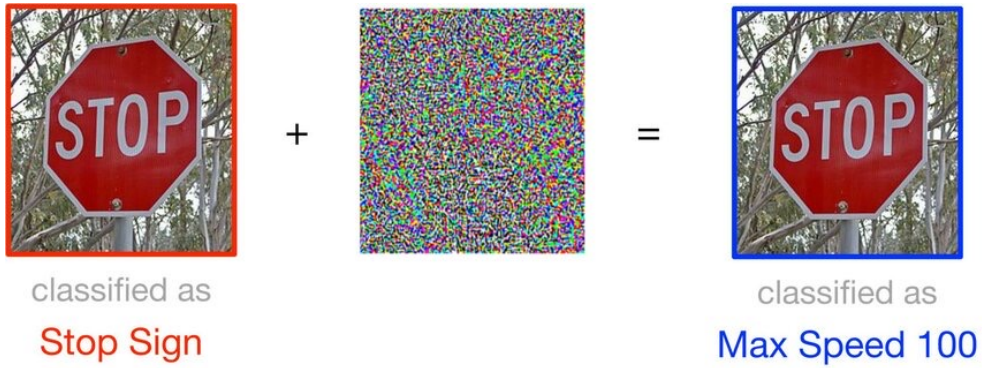
# Robustness: Agenda
## Adversarial Robustness

- Types of adversarial attacks

- Adversarial training



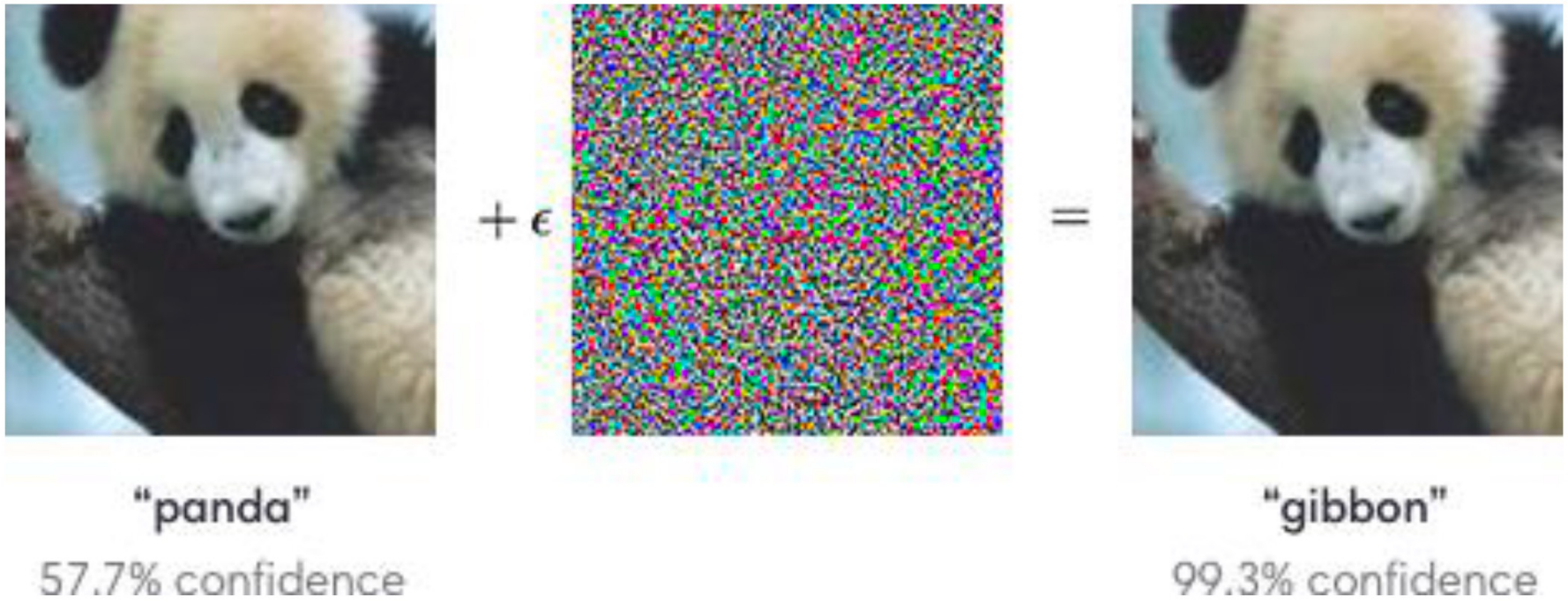

Szegedy et al., Intriguing Properties of Neural Networks, 2014




https://www.altacognita.com/robust-attribution/

# Adversarial Attacks and Robustness

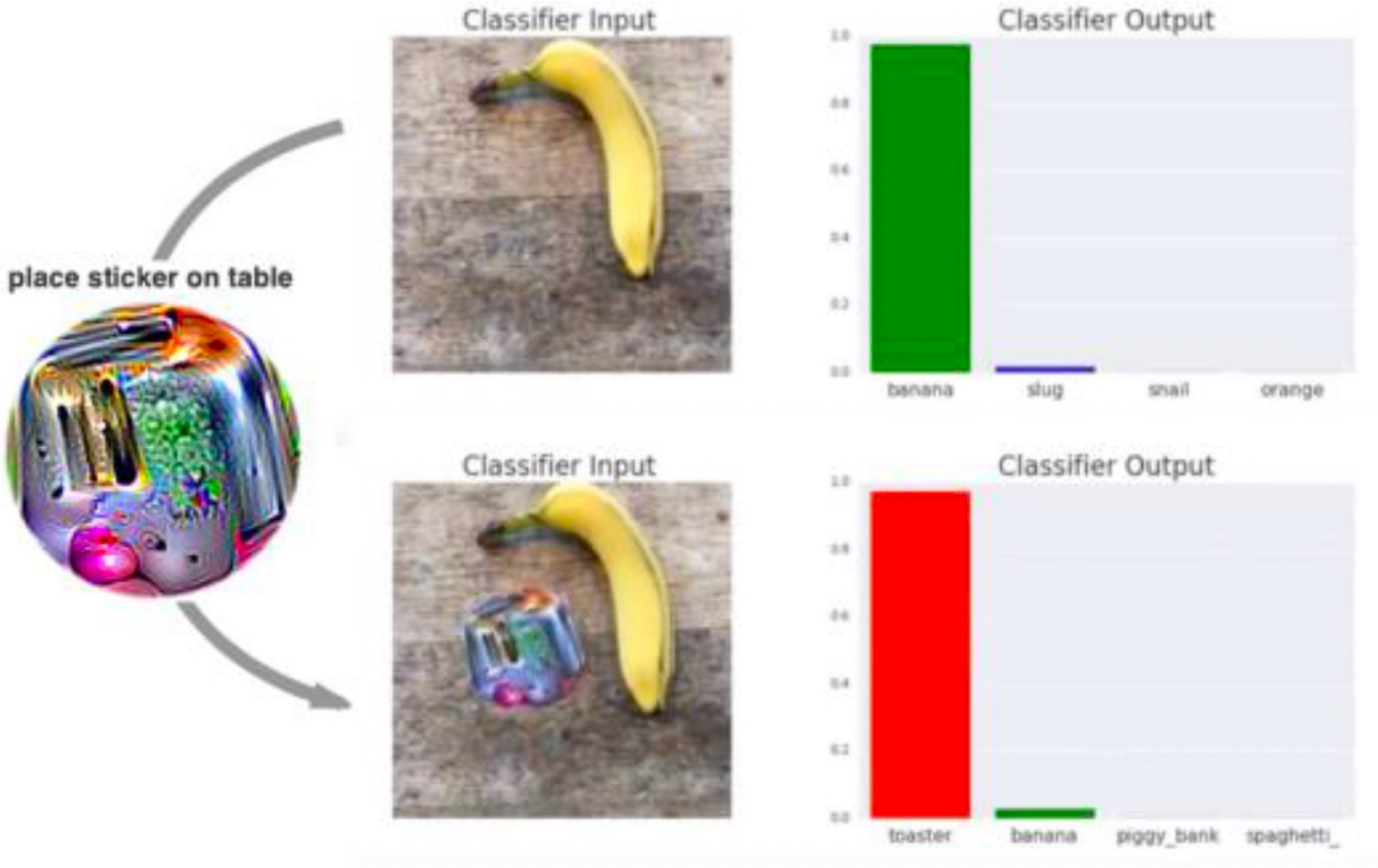
# Deep networks are “sensitive”




Szegedy et al., Intriguing Properties of Neural Networks, 2014

Image: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Adversarial Attacks Everywhere




Brown et al, 2017 “Adversarial Patch”

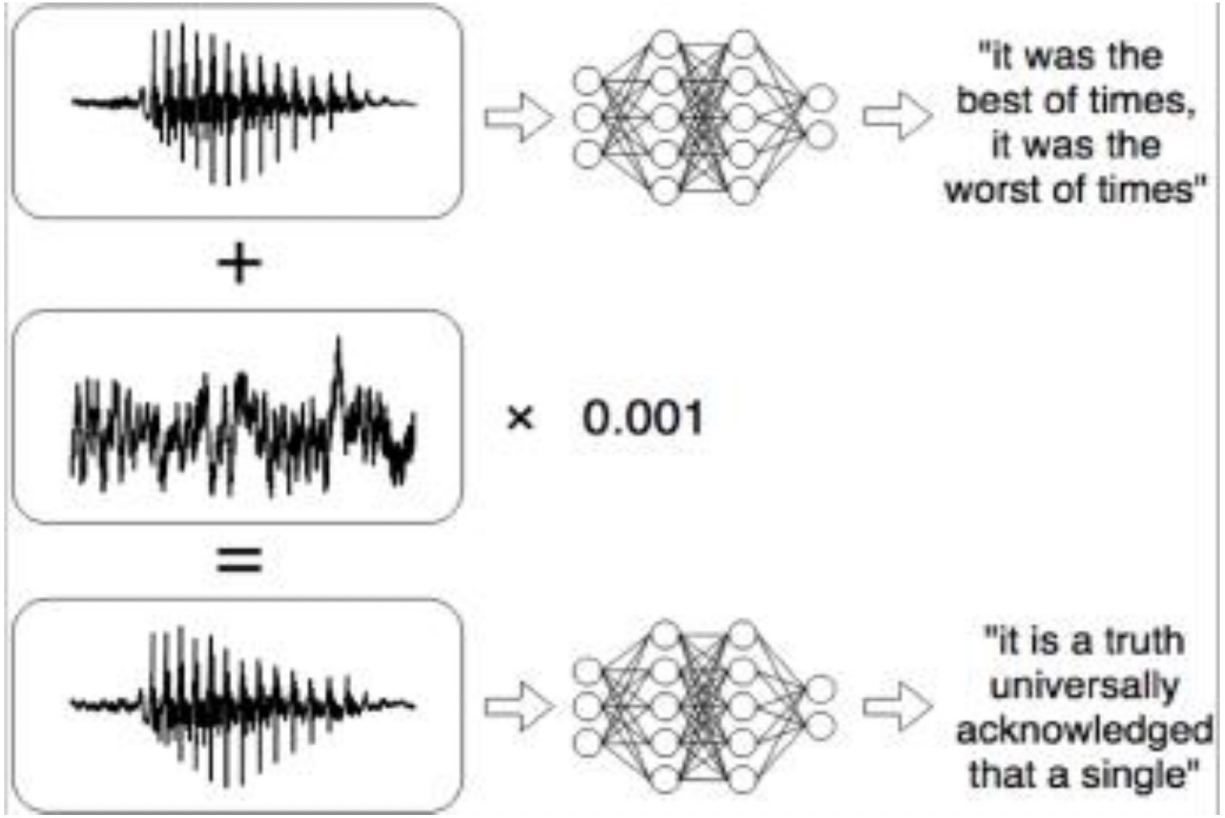
Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Adversarial Attacks Everywhere

| Label | Sentence |
|---|---|
| P | I am currently trying to give this company another chance. I have had the same scheduling experience as others have written about. Wrote to them today |
| N | I am currently trying to give this company another review. I have had the same dental experience about others or written with a name. Thanks to them today |

Hsieh et al. 2019 “Natural Adversarial Sentence Generation with Gradient based Perturbation”

# Adversarial Attacks Everywhere




Carlini, Wagner, 2018

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Adversarial Perturbations can be dangerous …

- **Task:**
  - Photo ID verification
  - Goal is to check whether uploaded photo matches a photo ID
- **Attack:**
  - User perturbs their image to match the photo in the ID
  - Challenge for machine learning in online identity verification!



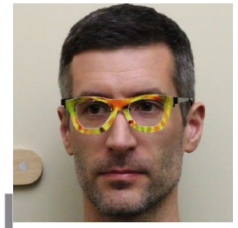

(Valid photo ID from Papesh 2018)

Wang, D., Yao, W., Jiang, T., Tang, G., & Chen, X. (2022). A survey on physical adversarial attack in computer vision. *arXiv preprint arXiv:2209.14262*.



Adversarial example against DNN is first revealed. [ICLR'14]

2016

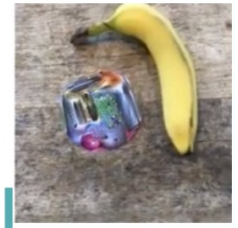


First physical attack against facial recognition system. [CCS'16]

First printed pixel-wise adversarial example. [ICLR'17]

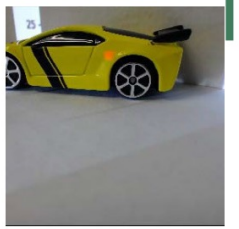


2017



First patch-based attack against image classifier. [NeurIPS'17]

First optical (laser) attack against image classifier. [AAAI-S'18]



2018

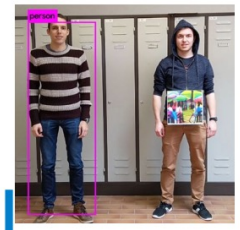


First 3D printed object with adversarial texture against image classifier. [ICML'18]

First patch-based attack against traffic sign recognition. [CVPR'18]




2019

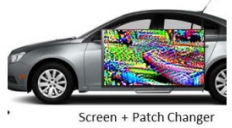


First patch-based attack on pedestrian detector. [CVPR-W'19]

First textured-based attack on vehicle detector. [ICLR'19]

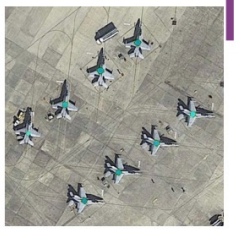


2020

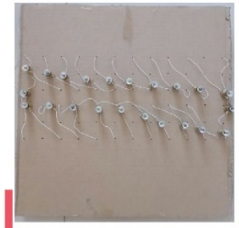



First dynamic patch-based attack against object detector. [ArXiv'20]

First patch-based attack on aerial image detector. [ArXiv'20]



2021



First physical attack with blub against infrared detector. [AAAI'20]

Frist neural renderer-based texture attack against object detector. [CVPR'21]



2022



First natural sticker attack against face recognition. [TPAMI'22]

First environment-driven optical attack against image classifier. [CVPR'22]



Wang, D., Yao, W., Jiang, T., Tang, G., & Chen, X. (2022). A survey on physical adversarial attack in computer vision. *arXiv preprint arXiv:2209.14262*.



Random



CAMOU [11]



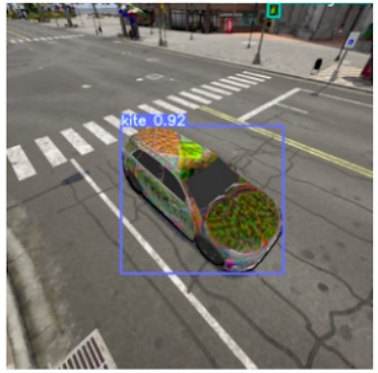

UPC [80]



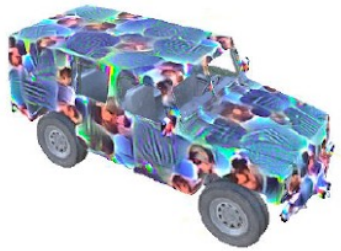

ER [159]



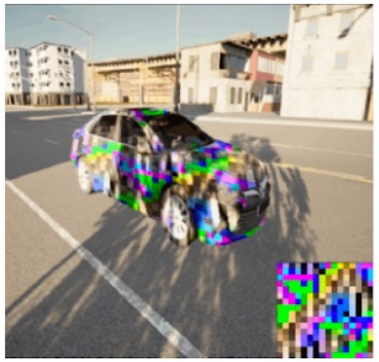
DAS [12]



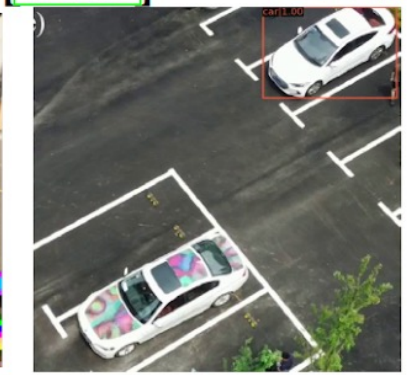

FCA [13]



CAC [36]



DTA [37]

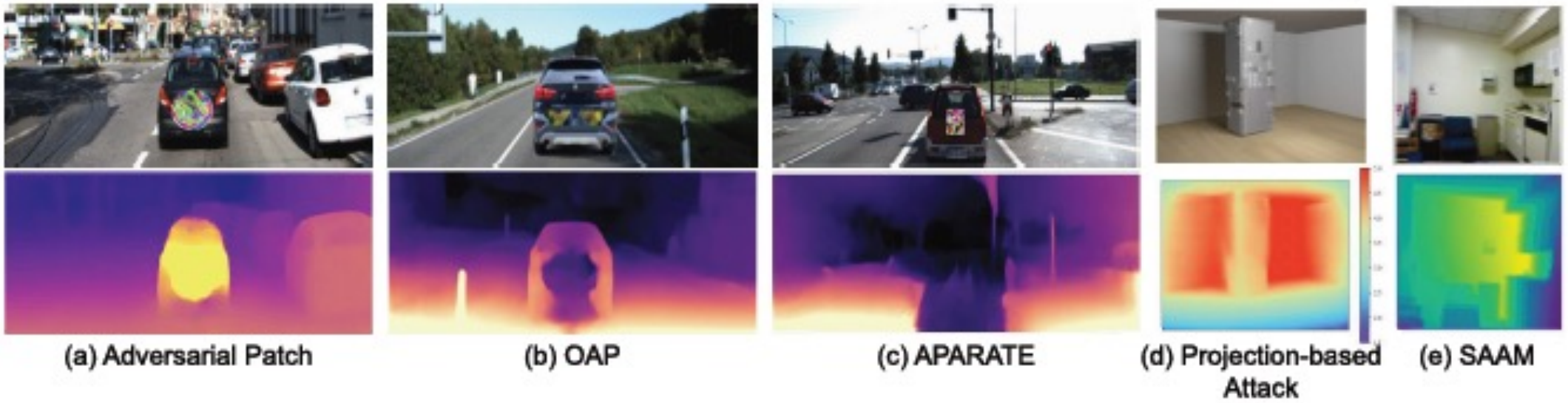


CAC [160]

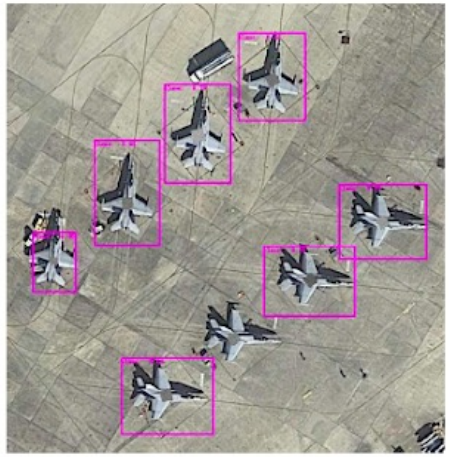
Guesmi, A., Hanif, M. A., Ouni, B., & Shafique, M. (2023). Physical adversarial attacks for camera-based smart systems: Current trends, categorization, applications, research challenges, and future outlook. IEEE Access.



Guesmi, A., Hanif, M. A., Ouni, B., & Shafique, M. (2023). Physical adversarial attacks for camera-based smart systems: Current trends, categorization, applications, research challenges, and future outlook. IEEE Access.

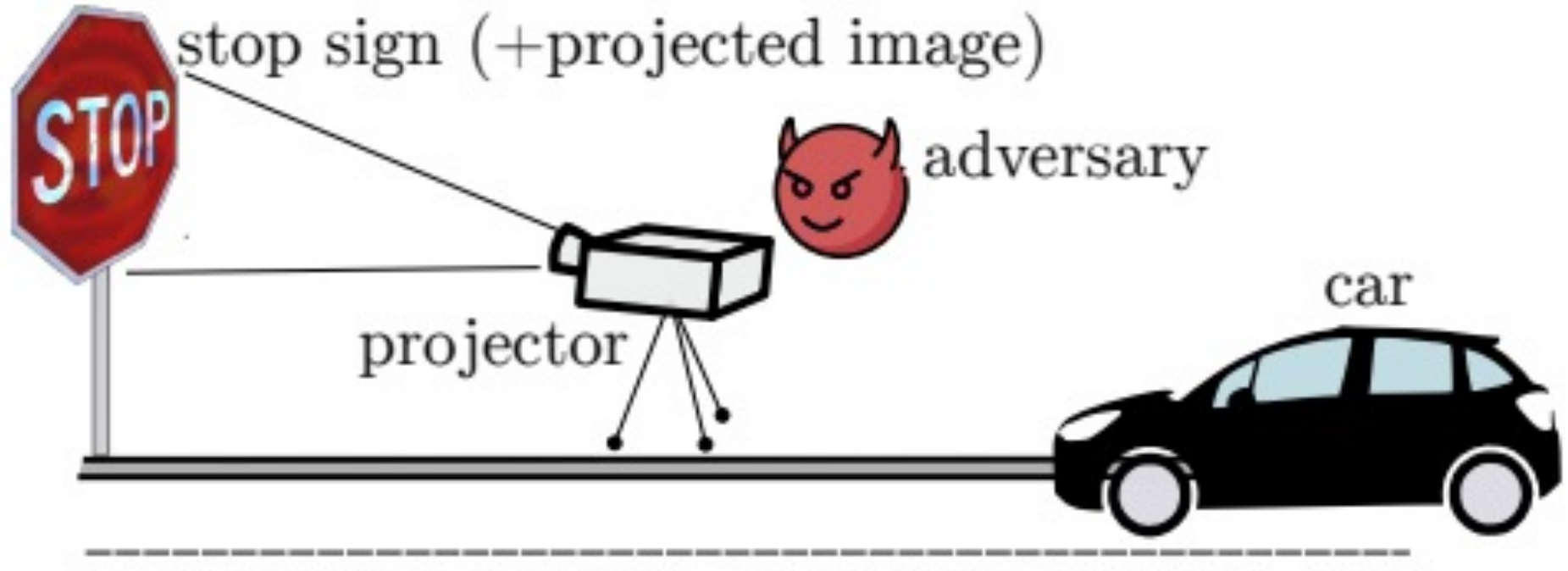


Wang, D., Yao, W., Jiang, T., Tang, G., & Chen, X. (2022). A survey on physical adversarial attack in computer vision. *arXiv preprint arXiv:2209.14262*.







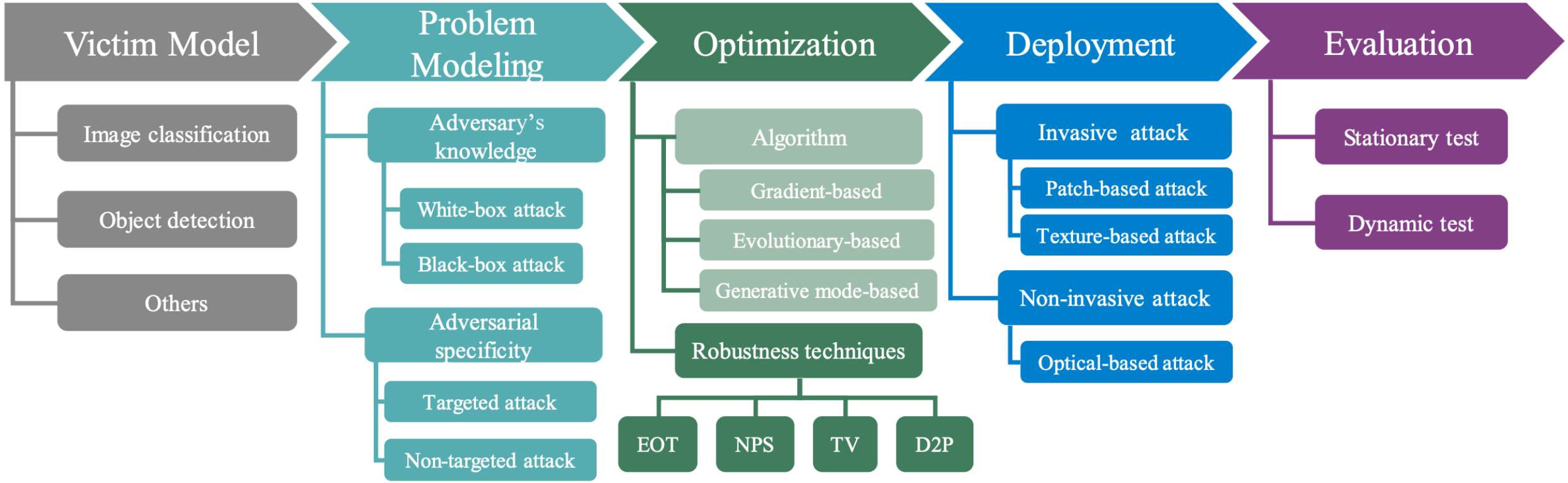

Lovisotto, G., Turner, H., Sluganovic, I., Strohmeier, M., & Martinovic, I. (2021). SLAP: Improving physical adversarial examples with {Short-Lived} adversarial perturbations. In 30th USENIX Security Symposium (USENIX Security 21) (pp. 1865-1882).

# Types of Attacks

- Digital vs Physical
  - Digital: Victim and attack are both soft
  - Physical: Victim and attack are both physical
- White-box vs Black-box
  - White-box: We have access to the model and its parameters (we can make forward and backward passes).
  - Black-box: We have access to only model predictions.
- Targeted vs Non-targeted
  - Targeted: The attack aims to obtain highest probability for a pre-determined target class.
  - Non-targeted: Any non-correct class is fine.

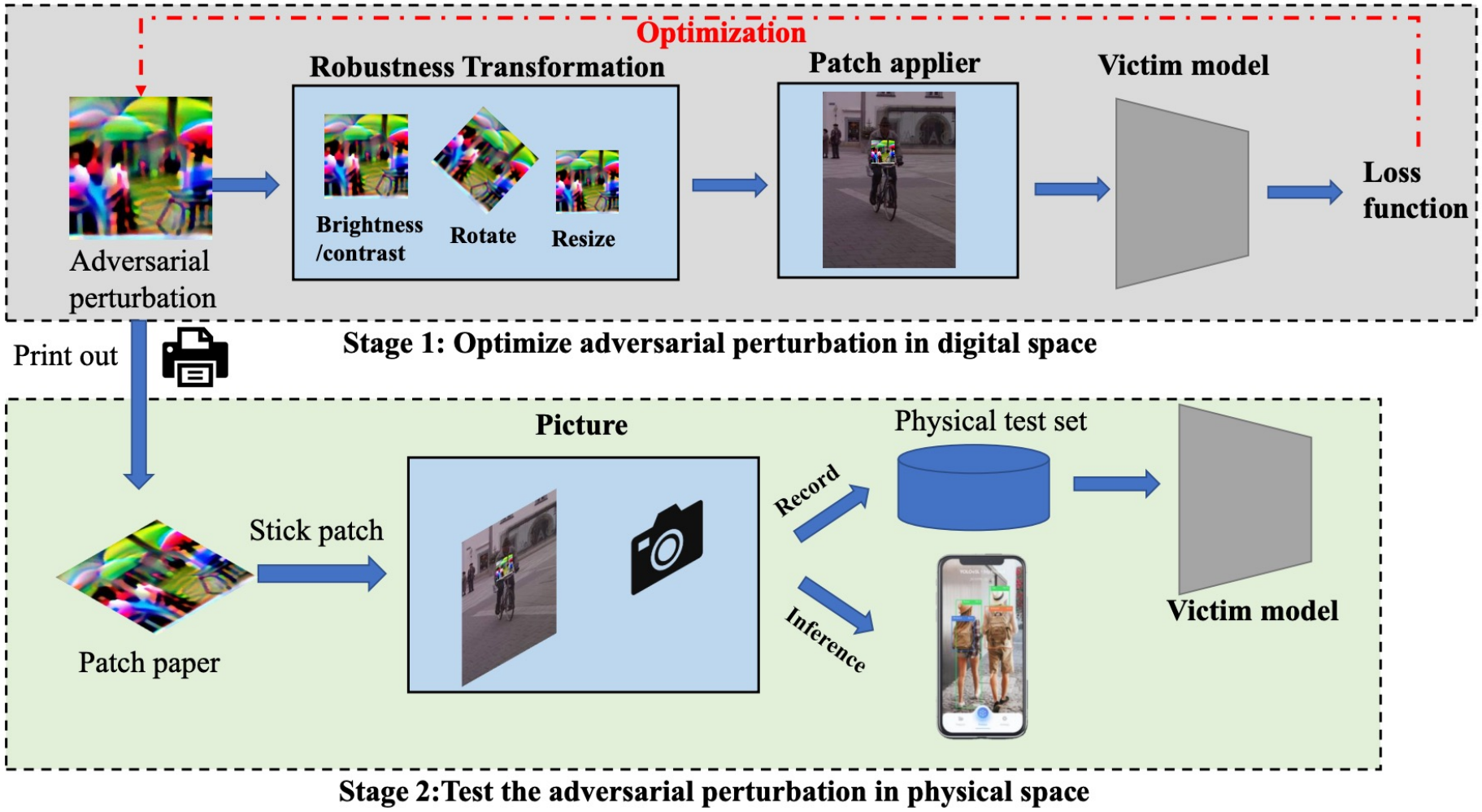
# Adversarial Attack Pipeline

Wang, D., Yao, W., Jiang, T., Tang, G., & Chen, X. (2022). A survey on physical adversarial attack in computer vision. *arXiv preprint arXiv:2209.14262*.



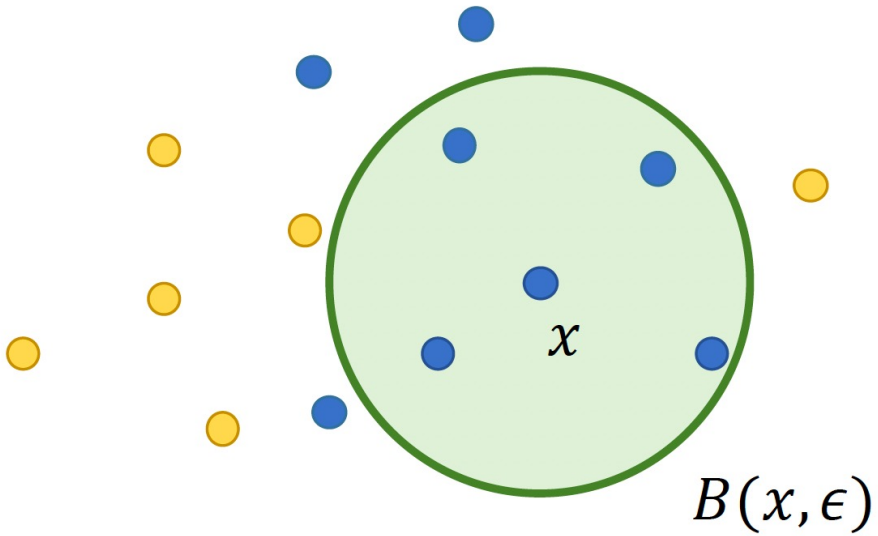
# Physical Attacks

Wang, D., Yao, W., Jiang, T., Tang, G., & Chen, X. (2022). A survey on physical adversarial attack in computer vision. *arXiv preprint arXiv:2209.14262*.

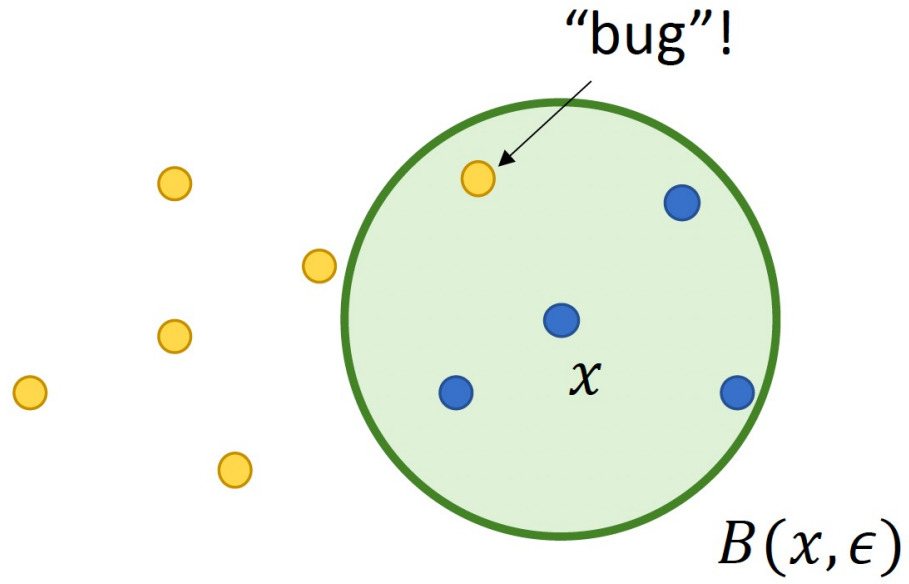


**robustness:** similar images ⇒ same label

**robustness:** $\|x - x'\|_\infty \leq \epsilon \Rightarrow$ same label



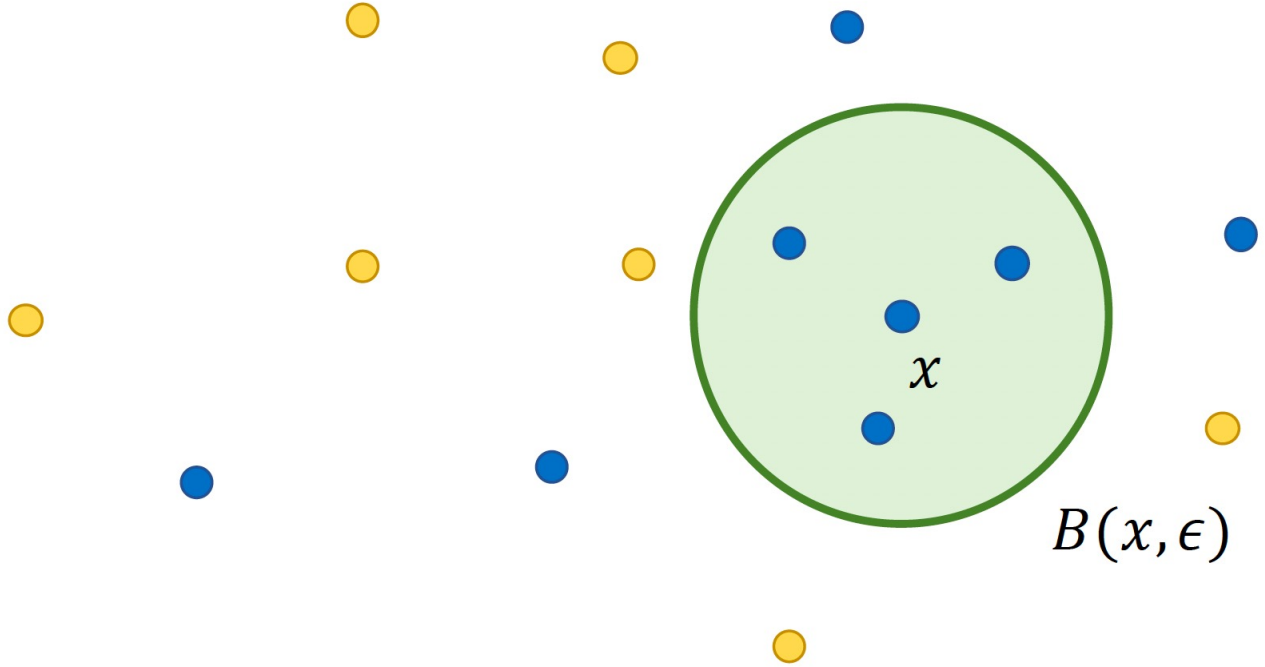

$\epsilon$-robust at $x$




not $\epsilon$-robust at $x$

# Is there a simple fix using data augmentation?

**Doesn't work to ensure robustness!**
**In theory as well as practice!!**




- Sample multiple points close to x
- Assign them same label as x
- Add them to training data set and retrain

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Generating Adversarial Examples

# Supervised Learning

- Given a model $f$ parameterized by θ
- $\text{Loss}(x, y;\ \theta)$ denotes the error of $f_\theta$ on input $x$ with respect to desired output $y$

- Learning as optimization:

  Given a training set of labeled input/output pairs $(x, y)$,

  find $\theta$ to minimize the average training loss

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Adversarial Example Computation

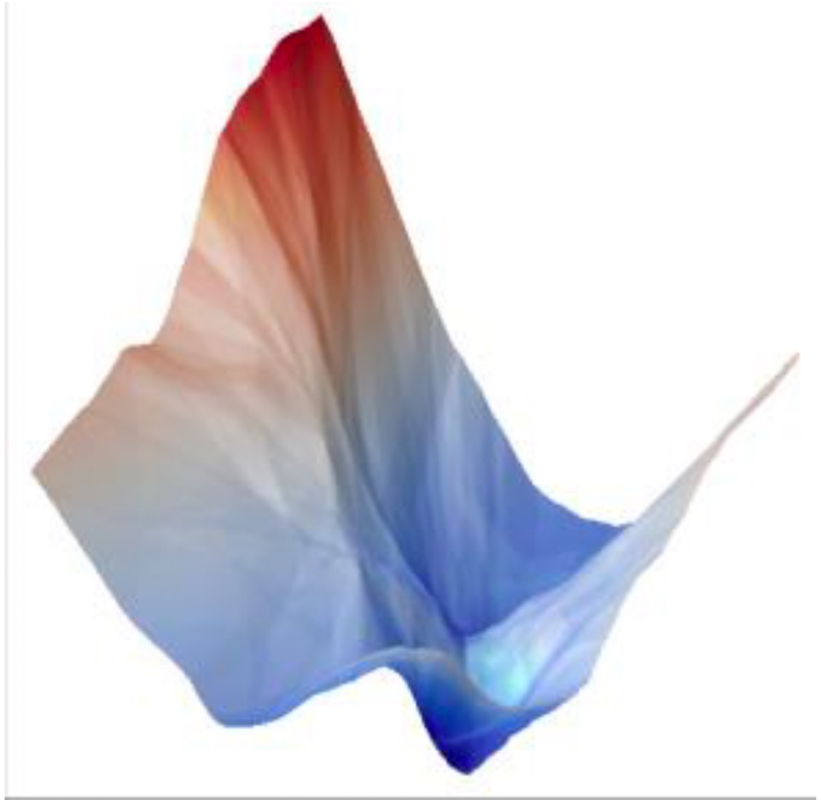
- Given a (trained) model $f$ with parameters $\theta$
- Fix input $x$ and corresponding output $\mathrm{y} = f_\theta(x)$
- Loss$(x + \delta, y;\ \theta)$ denotes the "change" in output with respect to δ–perturbation in input
- How can we formalize searching for adversarial example as optimization?

**Given a bound Δ on input perturbation,**

**find $0 < \delta < \Delta$ to maximize Loss$(x + \delta, y;\ \theta)$**

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

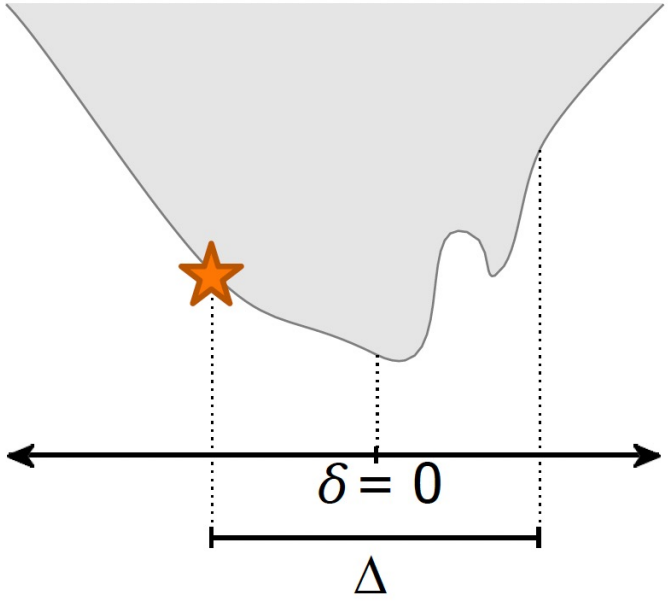
# Challenge: Complexity of model / loss function



$$\max_{\delta<\Delta} \text{Loss}(x+\delta, y;\ \theta)$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Solution: Local Search using Gradient Descent




$$\max_{\delta<\Delta} \text{Loss}(x+\delta, y;\ \theta)$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# How to implement desired gradient descent ?

- Key step in learning:

  Computing the gradient $\nabla_\theta$ Loss$(x, y;\ \theta)$ using backpropagation

- Question: How will you compute $\nabla_\delta$ Loss$(x + \delta, y;\ \theta)$?

Recall we are optimizing

$$\max_{\delta \in \Delta} \text{Loss}(x + \delta, y; \theta)$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# How to implement desired gradient descent ?

- Key step in learning:

  Computing the gradient $\nabla_\theta$ Loss$(x, y;\ \theta)$ using backpropagation

- Question: How will you compute $\nabla_\delta$ Loss$(x + \delta, y;\ \theta)$?

Recall we are optimizing

$$\max_{\delta \in \Delta} \text{Loss}(x + \delta, y; \theta)$$

- Question: To find the (locally) optimal value, is it ok to repeatedly update $x$ to $x\ +\ \alpha\ \nabla_\delta\ \text{Loss}(x + \delta, y;\ \theta)$, where $\alpha$ is the learning rate?

- No! We want $\delta\ <\ \Delta$, so this is an instance of "constrained optimization"

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Projected gradient descent

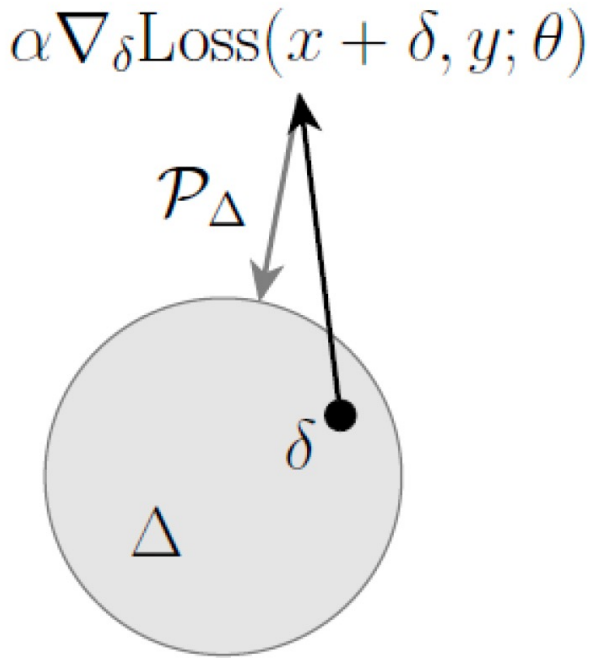
Recall we are optimizing

$$\max_{\delta \in \Delta} \text{Loss}(x + \delta, y; \theta)$$

We can employ a projected gradient descent method:

- take gradient step and
- project back into feasible set Δ

$$\delta := \mathcal{P}_{\Delta}[\delta + \nabla_{\delta} \text{Loss}(x + \delta, y; \theta)]$$




Source: Tutorial on Adversarial robustness by Kolter and Madry

# Fast Gradient Sign Method (FGSM)

Note that:

$$\|\mathbf{x}\|_\infty := \max_i |x_i|$$

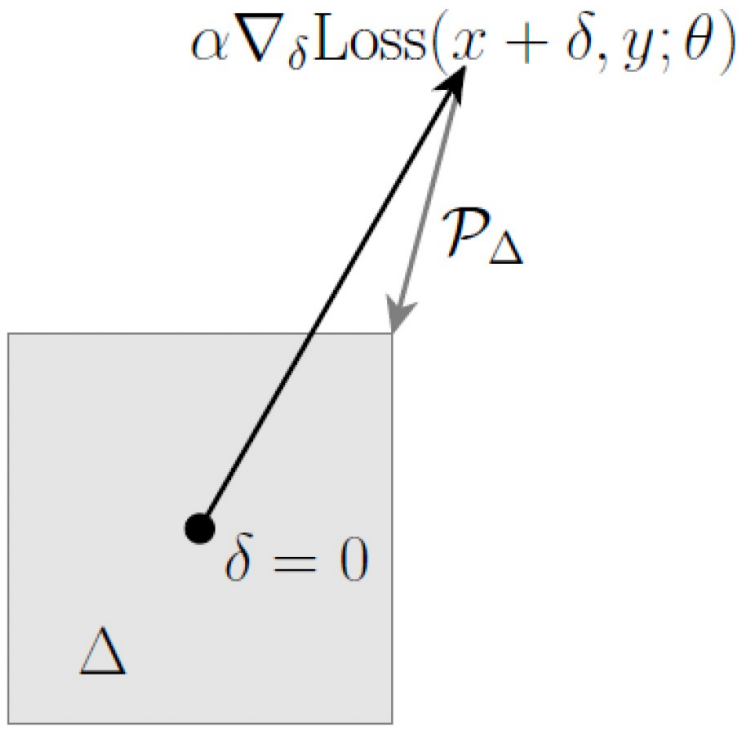
To be more concrete,

- take Δ to be the $\ell_\infty$ ball:

$$\Delta = \{\delta : \|\delta\|_\infty \leq \epsilon\}$$

- then, projection takes the form

$$P_\Delta(\delta) = \text{Clip}(\delta, [-\epsilon, \epsilon])$$




Source: Tutorial on Adversarial robustness by Kolter and Madry

# Fast Gradient Sign Method (FGSM)

Note that:

$$\|\mathbf{x}\|_\infty := \max_i |x_i|$$

To be more concrete,

- take Δ to be the $\ell_\infty$ ball:

$$\Delta = \{\delta : \|\delta\|_\infty \leq \epsilon\}$$

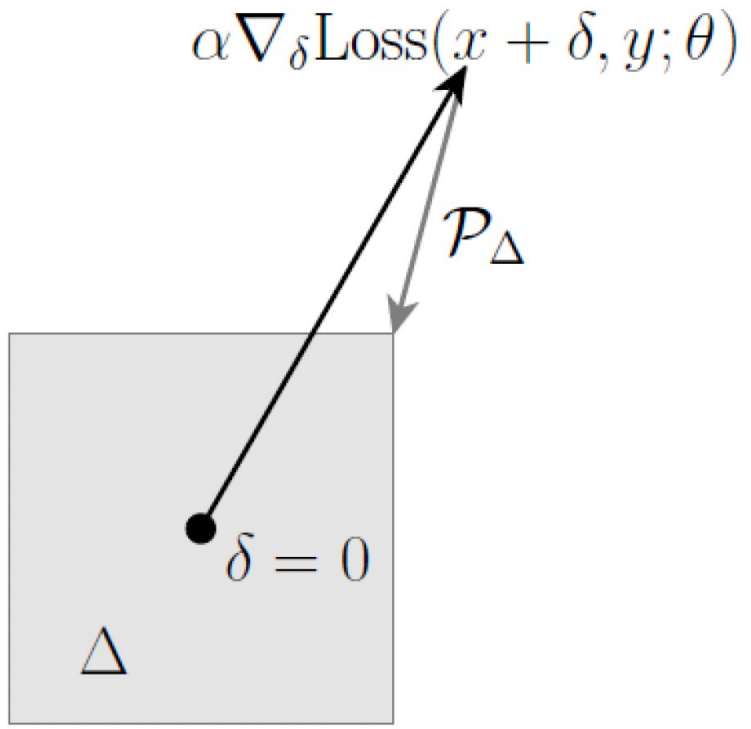
- then, projection takes the form

$$P_\Delta(\delta) = \text{Clip}(\delta, [-\epsilon, \epsilon])$$

As $\alpha \to \infty$, we always reach "corner" of the box, called fast gradient sign method (FGSM)

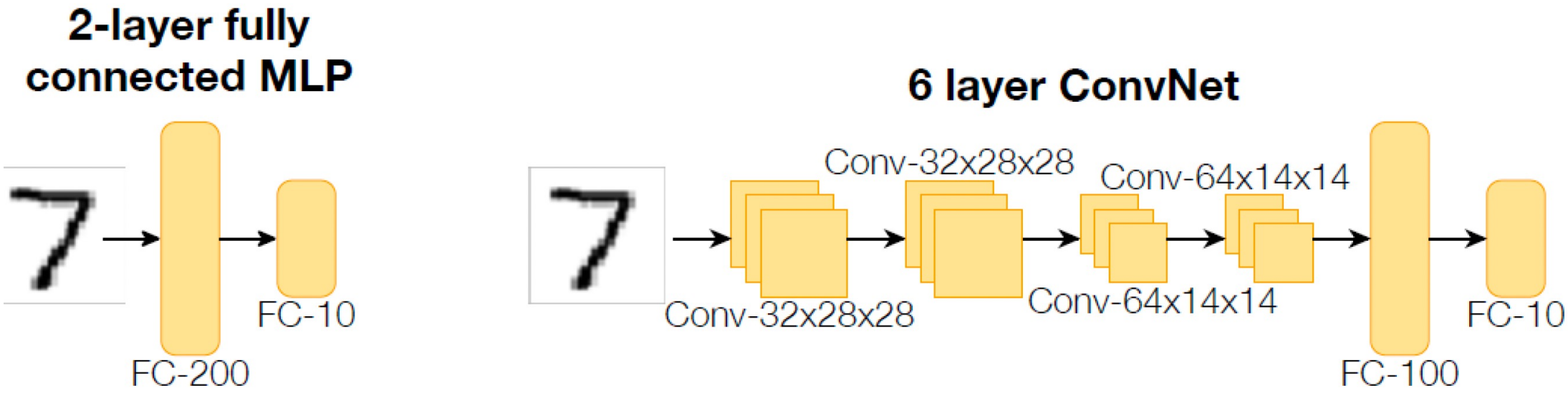
[Goodfellow et al., 2014]

$$\delta = \epsilon \cdot \text{sign}(\nabla_\delta \text{Loss}(x + \delta, y; \theta))$$




Source: Tutorial on Adversarial robustness by Kolter and Madry

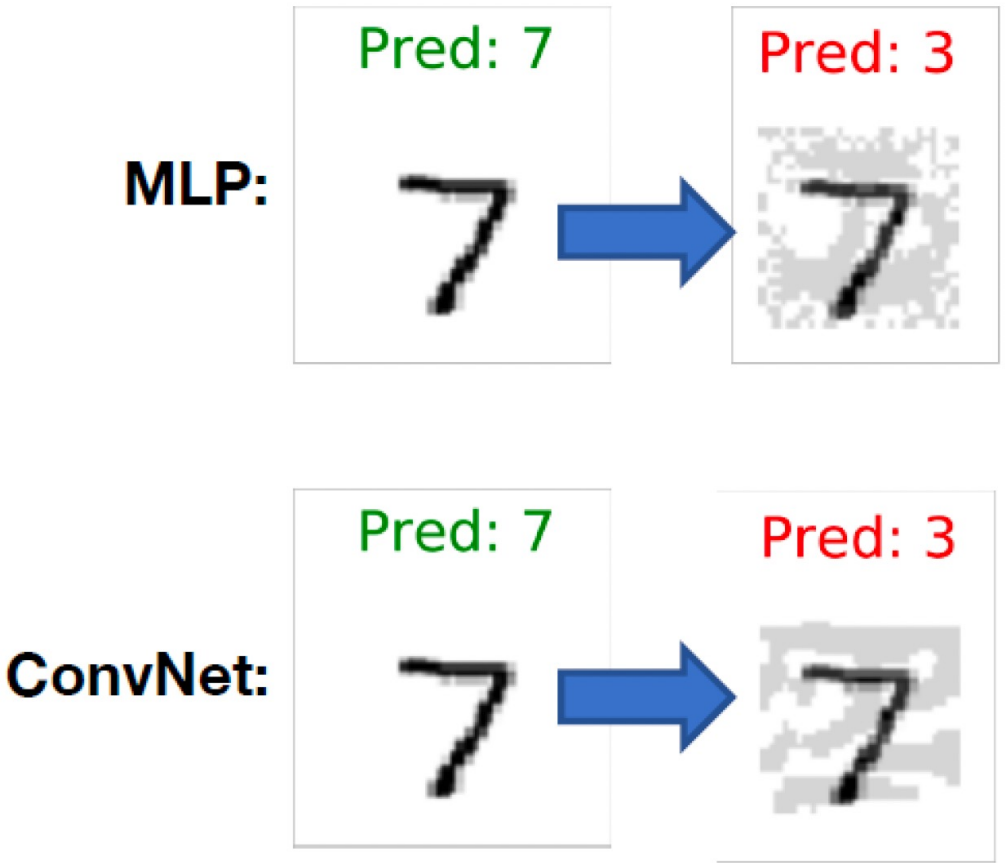
# Empirical Evaluation

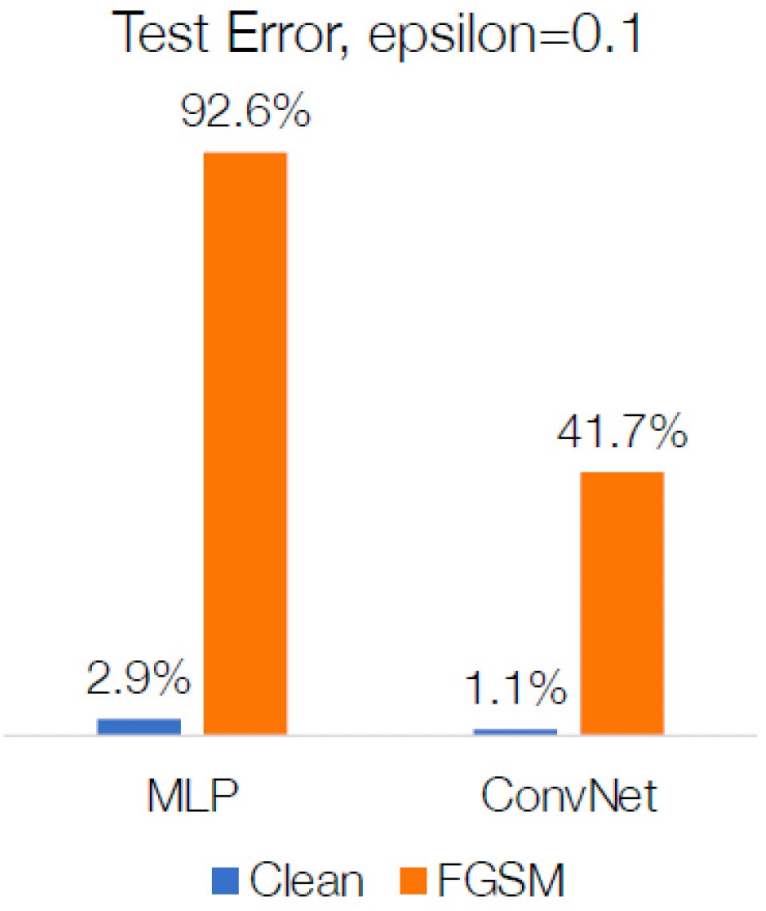



Source: Tutorial on Adversarial robustness by Kolter and Madry

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Evaluation of FGSM




Source: Tutorial on Adversarial robustness by Kolter and Madry

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

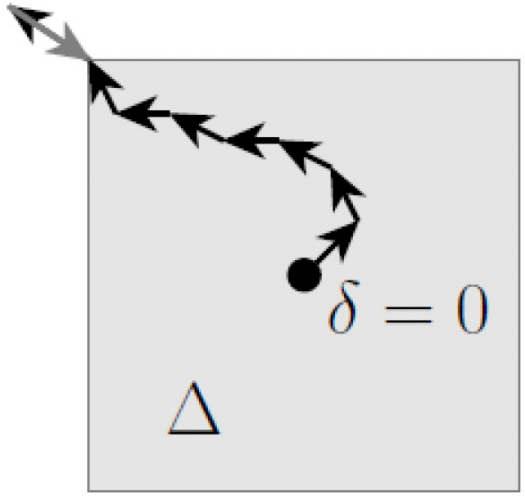
# Projected Gradient Descent

Projected gradient descent applied to $\ell_\infty$ ball, repeat:

$$\delta := \text{Clip}_\epsilon[\delta + \alpha \nabla_\delta J(\delta)]$$

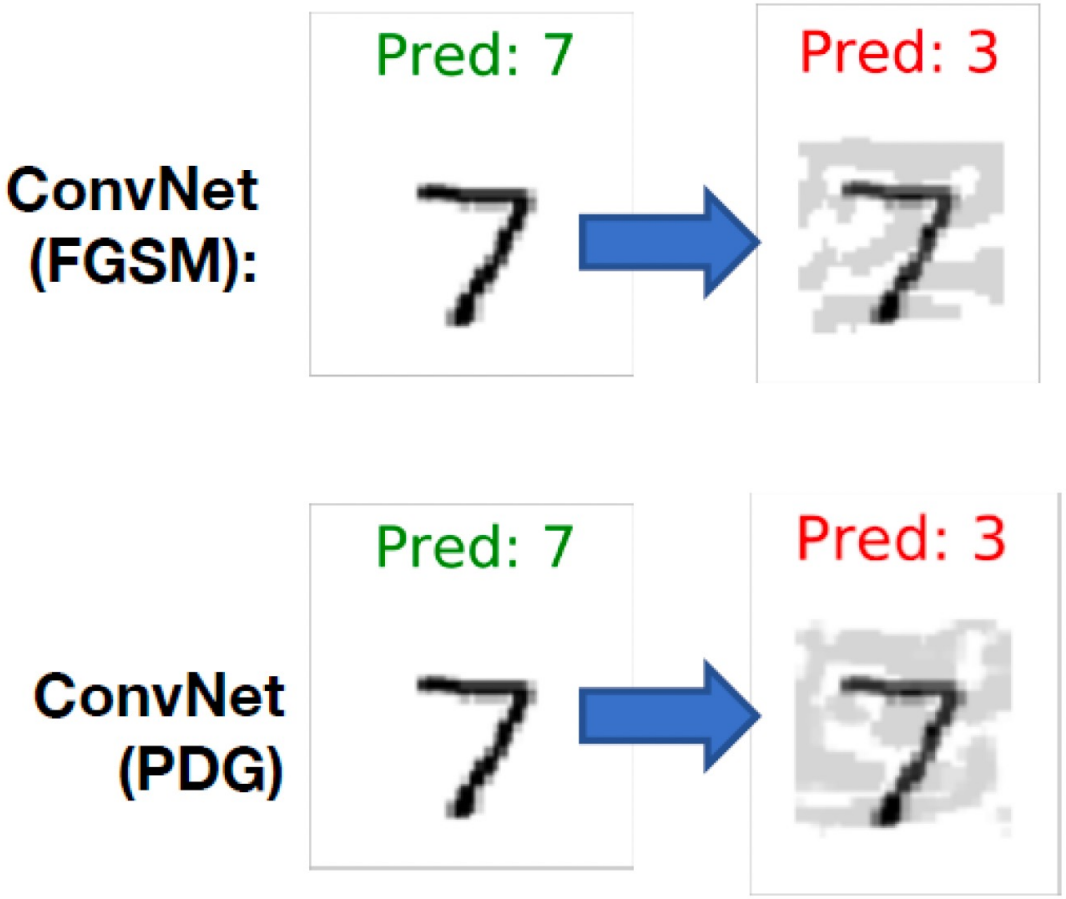
PGD vs FGSM:

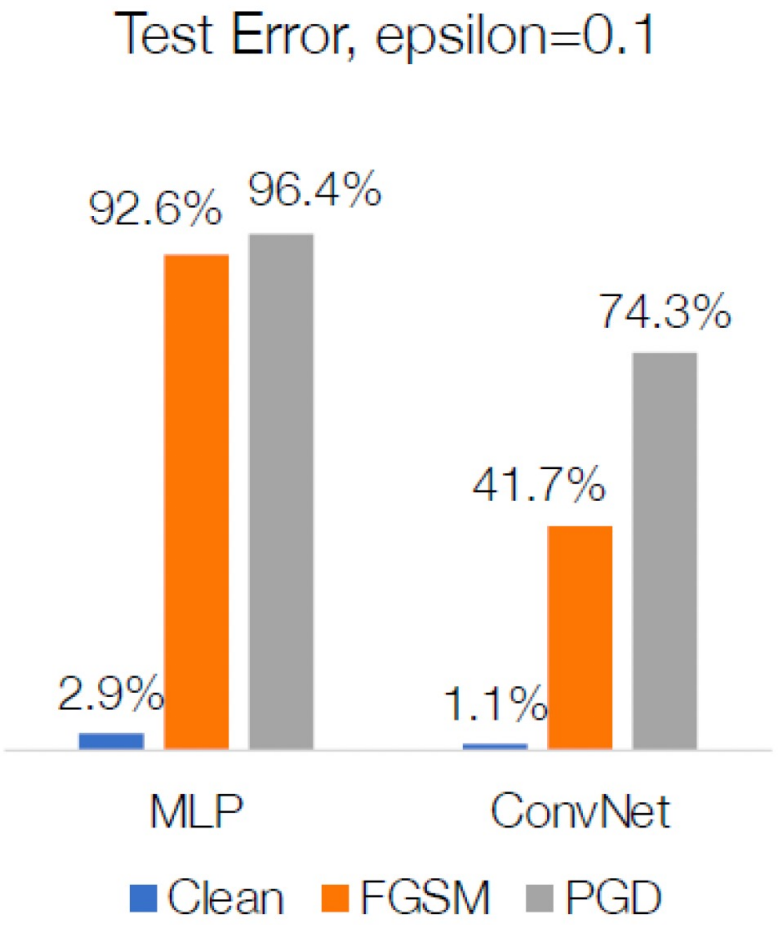
- PGD is slower than FGSM (requires multiple iterations),
- but typically able to find better optima




Source: Tutorial on Adversarial robustness by Kolter and Madry

# PGD Evaluation




Source: Tutorial on Adversarial robustness by Kolter and Madry

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Targeted Attack

Also possible to explicitly try to change label to a *particular* class

$$\max_{\delta \in \Delta} \left( \text{Loss}(x + \delta, y; \theta) - \text{Loss}(x + \delta, y_{\text{targ}}; \theta) \right)$$

Consider multi-class cross entropy loss

$$\text{Loss}(x + \delta, y; \theta) = \log \left( \sum_i \exp \; h_\theta(x + \delta)_i \right) - h_\theta(x + \delta)_y$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Targeted Attack

Also possible to explicitly try to change label to a *particular* class

$$\max_{\delta \in \Delta} \left( \text{Loss}(x + \delta, y; \theta) - \text{Loss}(x + \delta, y_{\text{targ}}; \theta) \right)$$

Consider multi-class cross entropy loss

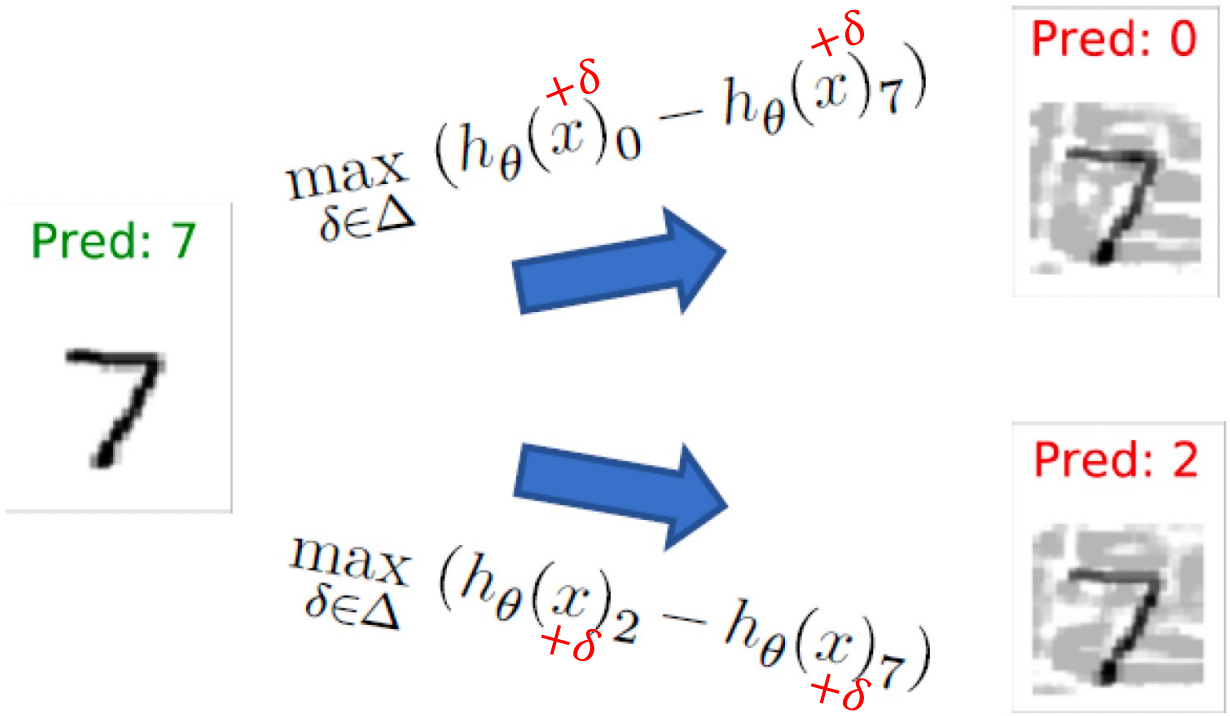
$$\text{Loss}(x + \delta, y; \theta) = \log \left( \sum_i \exp \; h_\theta(x + \delta)_i \right) - h_\theta(x + \delta)_y$$

Then, combining the two, we obtain:

$$\max_{\delta \in \Delta} \left( h_\theta(x + \delta)_{y_{\text{targ}}} - h_\theta(x + \delta)_y \right)$$

Source: Tutorial on Adversarial robustness by Kolter and Madry

# Targeted Attack Example




Source: Tutorial on Adversarial robustness by Kolter and Madry

# Alternative ways to solve the optimization problem

- Goal: Solve $\max_{\delta<\Delta}$ Loss$(x+\delta, y;\ \theta)$
- Another approach: encode the problem using constraints and use specialized and optimized constraint solvers (e.g., ReluPLEX)
- Another approach: Use convex relaxation for approximate solving
- We will revisit when we discuss verification/certification for robustness

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Beyond adding noise to images

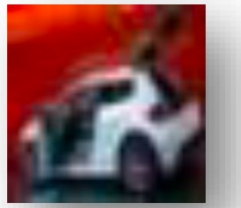
- Adversarial patches: Given a “patch” p, find an optimal position within given image $x$ so as to maximize the loss on the augmented image

- Text substitution: Given a set of allowed substitutions (e.g. words by their synonyms) find the modified sentence to maximize the loss

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Geometric Transformations

x



Rotate x between
$\alpha^{\circ}$ and $\beta^{\circ}$

$$x_0 = f_0(\alpha, \beta)$$
$$x_1 = f_1(\alpha, \beta)$$
.....



$perturbation\ set\ \alpha = -10^{\circ}$ and $\beta = 10^{\circ}$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Intriguing Properties of Neural Networks

- Adversarial examples can be computed efficiently using Fast Gradient Sign Method
- On image classification benchmarks, adversarial examples are so close to original examples that the difference is imperceptible to human eye
- Same adversarial example is often misclassified by alternative classifiers with different architectures or trained using different data set!

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Adversarial Training

# Adversarial Training

- Given a model $f$ parameterized by $\theta$
- Loss$(x, y;\ \theta)$ denotes the error of $f_\theta$ on input x with respect to desired output y
- Given training set S of labeled input/output examples $(x, y)$
- Goal: Account for adversarial examples during learning (update of parameters $\theta$)
- Adversarial training as optimization:

$$\min_{\theta} \sum_{x,y \in S} \max_{\delta \in \Delta} \text{Loss}(x + \delta, y; \theta)$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# MinMax Optimization

$$\min_{\theta} \sum_{x,y \in S} \max_{\delta \in \Delta} \text{Loss}(x + \delta, y; \theta)$$

How to obtain optimal $\theta$ by modifying gradient descent?

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Danskin's Theorem for solving MinMax problems

A fundamental result in optimization:

$$\nabla_\theta \max_{\delta \in \Delta} \text{Loss}(x+\delta, y; \theta) = \nabla_\theta \text{Loss}(x+\delta^\star, y; \theta)$$

where $\delta^\star = \arg\max_{\delta \in \Delta} \text{Loss}(x+\delta, y; \theta)$

Note:

- This assumes that we are computing δ* exactly,
- but we are not!

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Adversarial Training Algorithm

Repeat:

1. Select a minibatch $B$
2. For each $(x, y)$ in $B$, compute the adversarial example $\delta^\star(x)$

Recall FGSM method of steepest descent to compute adversarial examples:

$$\delta^* = \epsilon \cdot \text{sign}(\nabla_\delta \text{Loss}(x + \delta, y; \theta))$$

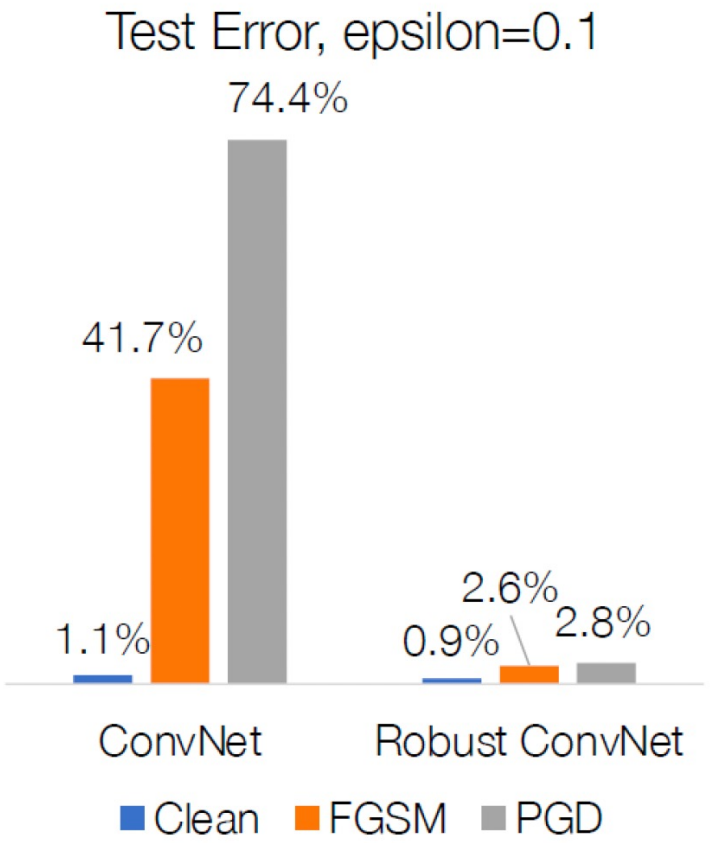
3. Update parameters:

$$\theta := \theta - \frac{\alpha}{|B|} \sum_{x,y \in B} \nabla_\theta \text{Loss}(x + \delta^\star(x), y; \theta)$$

Note: in practice, one can mix standard updates and adversarial updates

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Empirical Evaluation of Robust Training




Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Beyond Empirical Defenses

- Adversarial training improves robustness empirically
- But adversarial example is only one type of attack, new attacks need new defenses
- Certified robustness: Can we get mathematical guarantees of robustness ?
- Certified Robustness via randomized smoothing [Cohen et al; 2019]

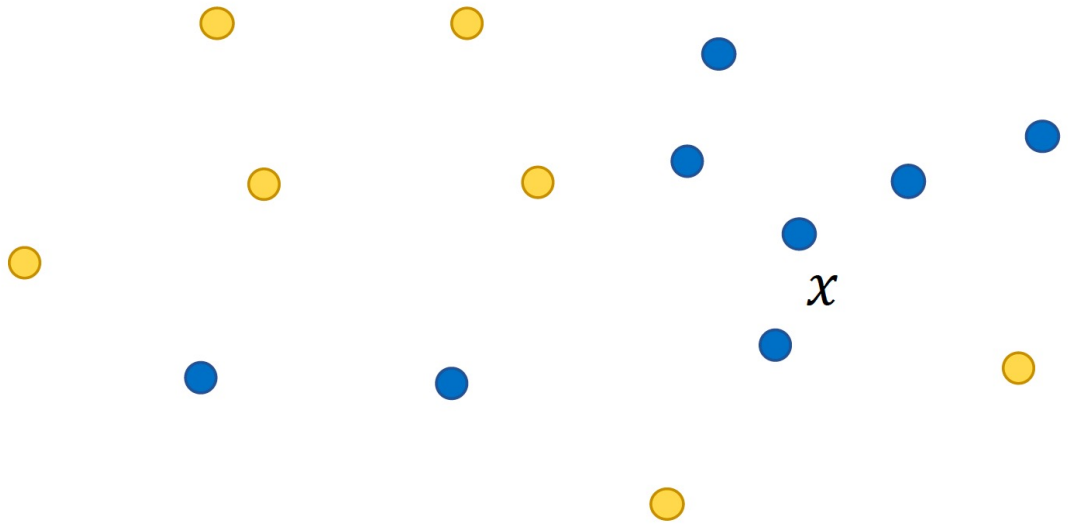
Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Other types of attacks

- **Data Poisoning Attacks:** The attacker injects malicious / mislabeled data into the training set.
- **Model Inversion / Membership Inference (Privacy Attacks):** The attacker analyzes the model's outputs to reconstruct sensitive data used during training or to determine if a specific individual's data was part of the training set.
- **Model Extraction (Stealing):** An attacker repeatedly queries a deployed model to train a surrogate model, effectively stealing the intellectual property.
- **Sponge / Denial of Service (DoS) Attacks:** The attacker crafts inputs specifically designed to maximize the model's computation time or energy consumption, degrading system availability.

# Certified Adversarial Robustness

# Smoothing of a given classifier, informally

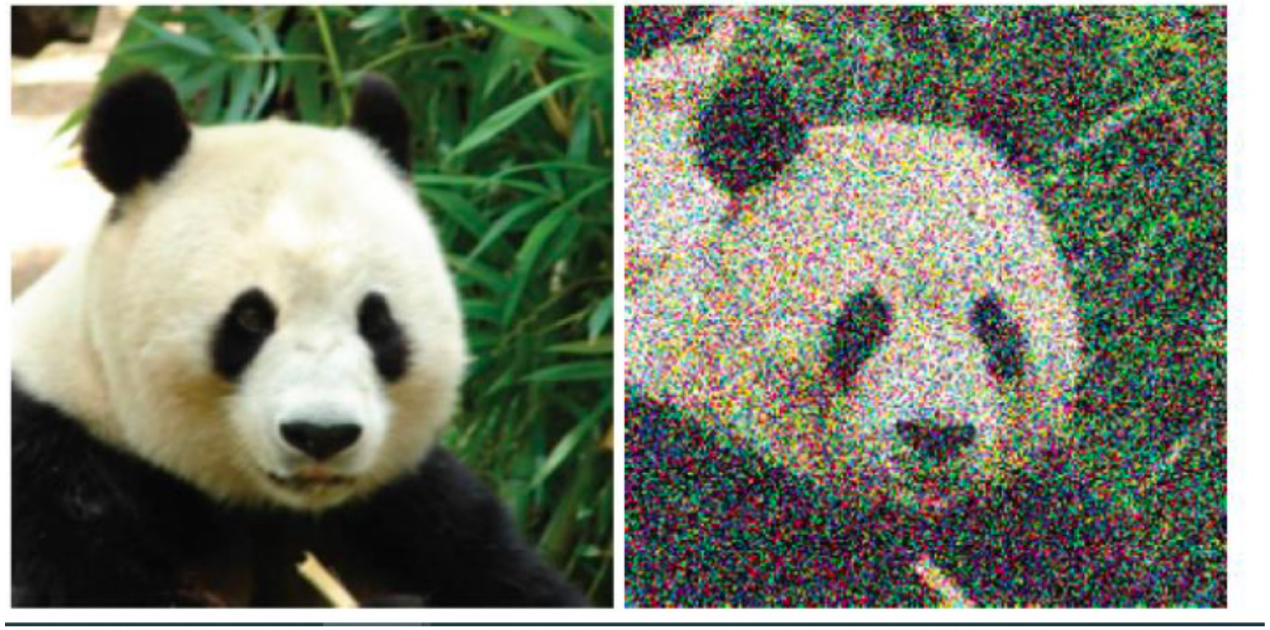



- Sample multiple perturbations $x$' of $x$
- Compute the label $f(x')$ for each variant
- Set $g(x)$ to the majority vote

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Creating Random Perturbations

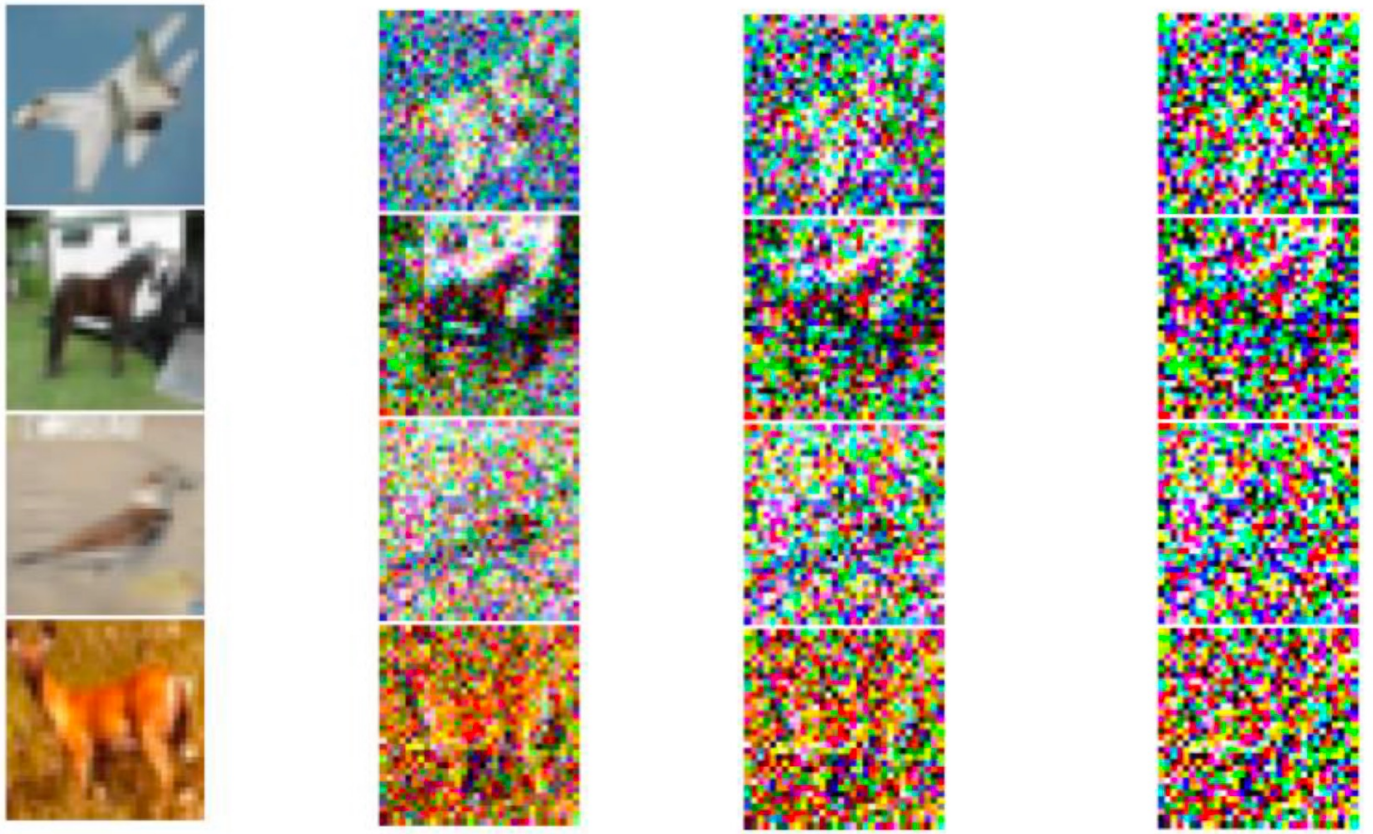
- Given an input $x$, consider inputs $x + \eta$,
  - $\eta \sim \mathcal{N}(0, \sigma^2 I)$
  - In other words: $\eta$ is noise sampled from Gaussian distribution with mean 0 and variance $\sigma^2$



Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Examples of noisy images from CIFAR-10

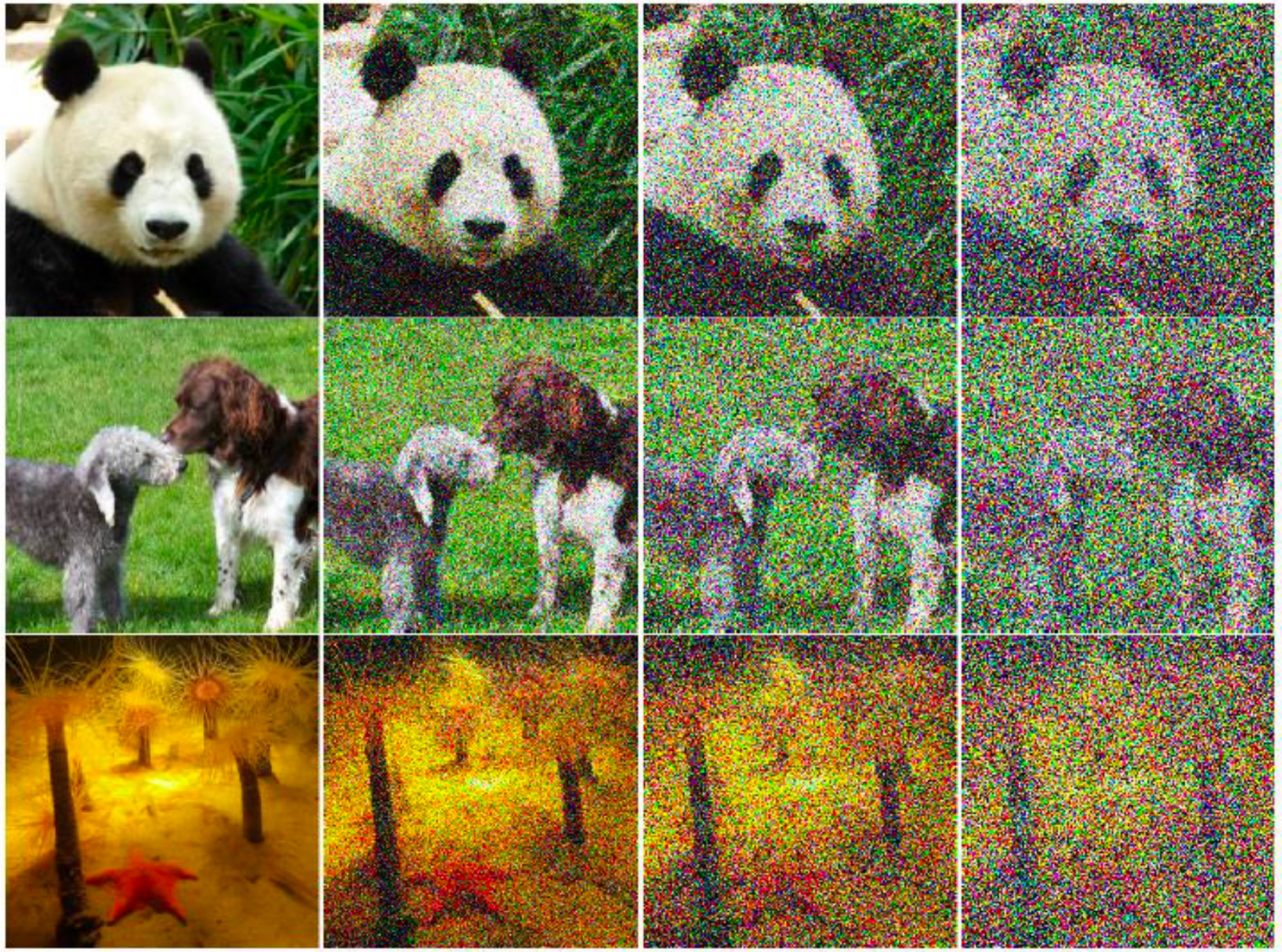
- with varying levels of Gaussian noise $\mathcal{N}(0, \sigma^2 I)$ from $\sigma = 0$ to $\sigma = 1$




Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

Examples of noisy images from ImageNet

- with varying levels of Gaussian noise $\mathcal{N}(0, \sigma^2 I)$ from $\sigma = 0$ to $\sigma = 1$




Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Smoothed classifier

- Given a base classifier $f$, its smoothed version $g$ maps an input $x$ to the majority prediction of $f$ on many Gaussian-perturbed images $x + \eta$:

Voting

$$g(x) = \arg\max_y \mathbb{P}_\eta[f(x+\eta) = y]$$

Probability that the base classifier estimates class y for the Gaussian-perturbed image. Can be estimated with N perturbations as follows:

$$\hat{\mathbb{P}}_\eta[f(x+\eta) = y] \approx \frac{\text{Count of times } f(x+\eta_i) = y}{N}$$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Estimation by Monte Carlo Sampling

To design a smoothed classifier $g$ at the input sample $x$ requires to identify the most likely class $\hat{c}_A$ returned by the base classifier $f$ on noisy images:

- Step 1: create *n* versions of $x$ corrupted with Gaussian noise $\eta \sim \mathcal{N}(0, \sigma^2 I)$
- Step 2: evaluate the predictions by base classifier for corrupted images, $f(x + \eta)$
- Step 3: identify the top two classes $\hat{c}_A$ and $\hat{c}_B$ with the highest number of predictions on $f(x + \eta)$
- Step 4:

if $n_A > n_B$ => return $\hat{c}_A$ as the prediction by $g(x)$

Otherwise, if $n_A - n_B < \alpha$ => Abstain from making a decision

# predictions by $f$ for the top class $\hat{c}_A$

# predictions by $f$ for the top class $\hat{c}_B$

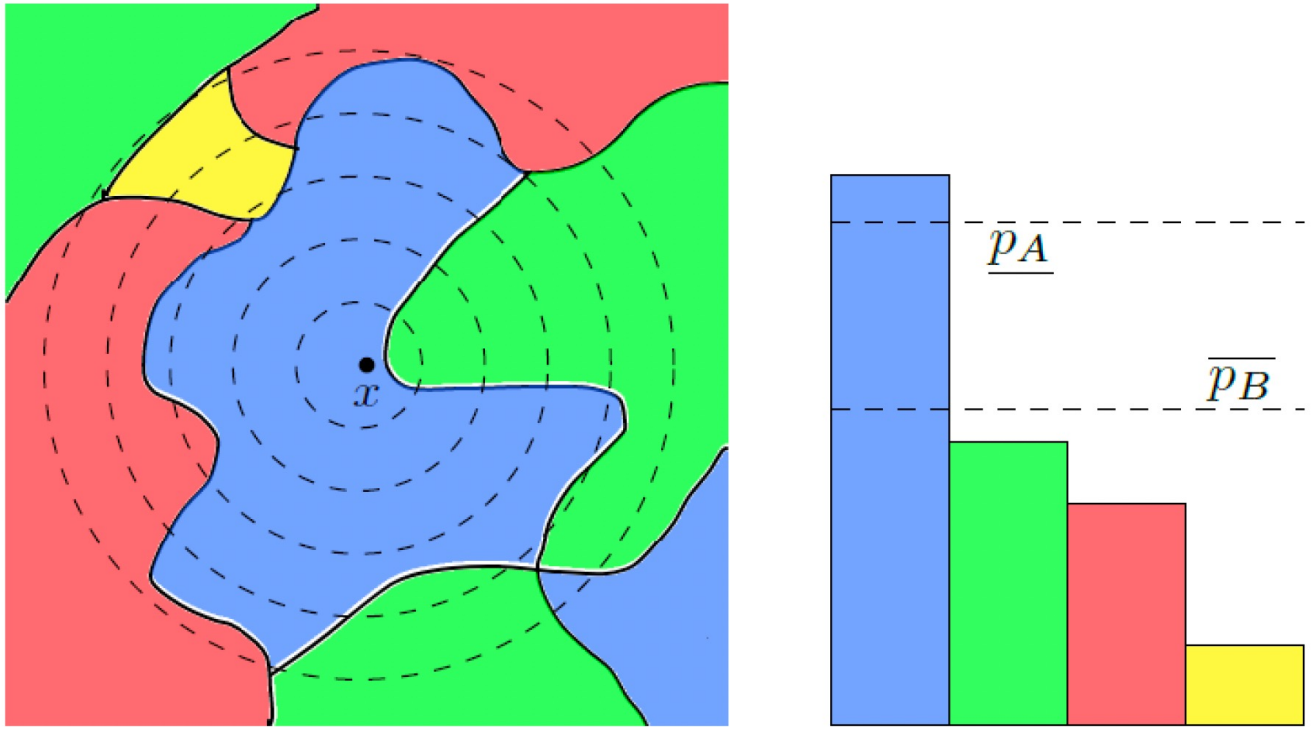
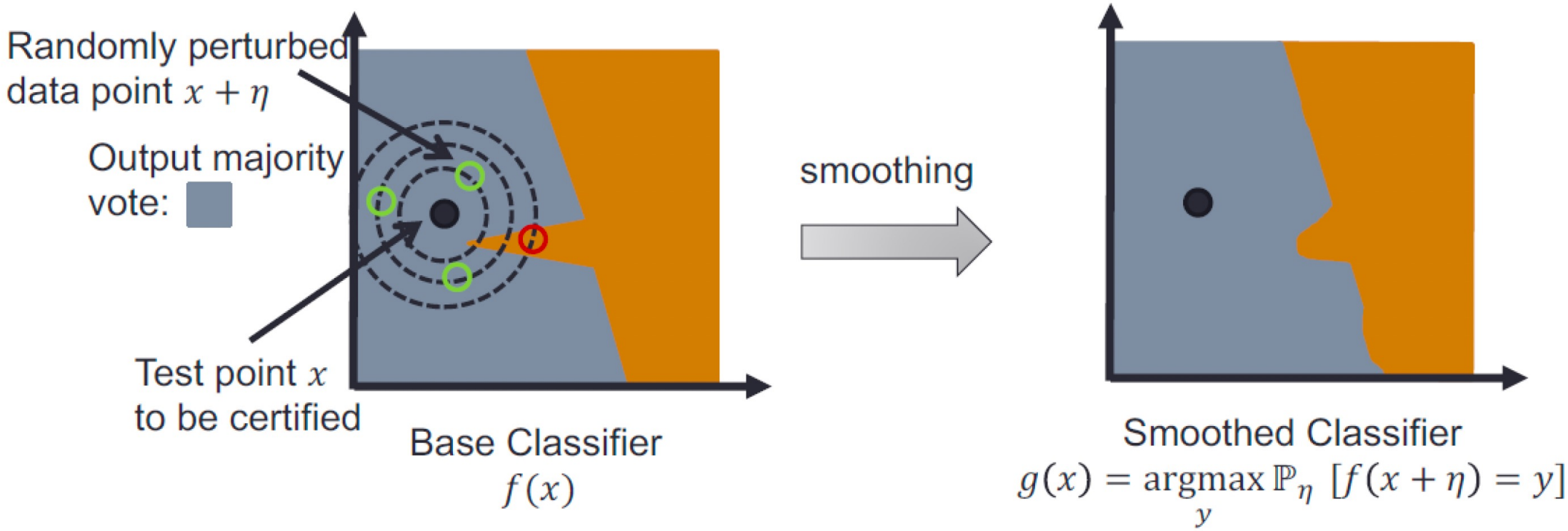



*Figure 1.* Evaluating the smoothed classifier at an input $x$. **Left**: the decision regions of the base classifier $f$ are drawn in different colors. The dotted lines are the level sets of the distribution $\mathcal{N}(x, \sigma^2 I)$. **Right**: the distribution $f(\mathcal{N}(x, \sigma^2 I))$. As discussed below, $\underline{p_A}$ is a lower bound on the probability of the top class and $\overline{p_B}$ is an upper bound on the probability of each other class. Here, $g(x)$ is "blue."

Cohen et al., Certified Adversarial Robustness via Randomized Smoothing, ICML 2019.

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

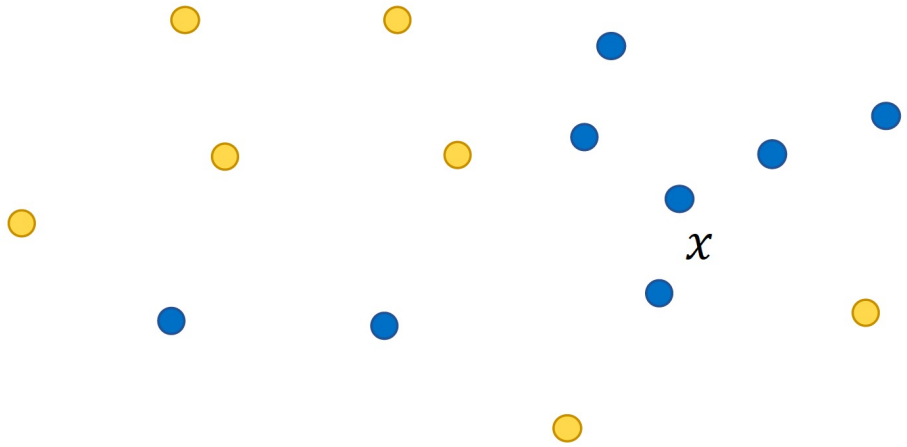
# Illustrating effect of smoothing




Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Randomized Smoothing

- Method works for an arbitrary $f$, including complex neural networks
- The smoothed version $g$ of a given classifier $f$ turns out to be empirically robust
- The bound $\Delta$ on adversarial robustness radius is related to the parameter $\sigma$ in Gaussian noise
- Intuitively: large random noise can be used to drown out small adversarial perturbation




Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Randomized Smoothing

- Method works for an arbitrary $f$, including complex neural networks
- The smoothed version $g$ of a given classifier $f$ turns out to be empirically robust
- The bound $\Delta$ on adversarial robustness radius is related to the parameter $\sigma$ in Gaussian noise
- Intuitively: large random noise can be used to drown out small adversarial perturbation
- Key question: can one establish this relationship provably?

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Randomized Smoothing Guarantee

Recall:

$$g(x) = \underset{y}{\operatorname{argmax}}\, \mathbb{P}_\eta\, [f(x+\eta) = y]$$

**Certified robust radius by [Cohen et al.'19] (for binary classification)**

- Given any input $x \in \mathbb{R}^d$, let
  - $\eta$ be Gaussian noise $\mathcal{N}(0, \sigma^2 I)$
  - $p = \max_y \mathbb{P}_\eta[f(x+\eta) = y]$ => Confidence of majority vote

- Then $g(x) = g(x+\delta)$ for any $\delta$ such that $\|\delta\|_2 \leq \Phi^{-1}(p)\sigma$,
  - where $\Phi$ is CDF of standard Gaussian.

Computable certified radius for *x*

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

WILL NOT BE INCLUDED IN THE EXAM

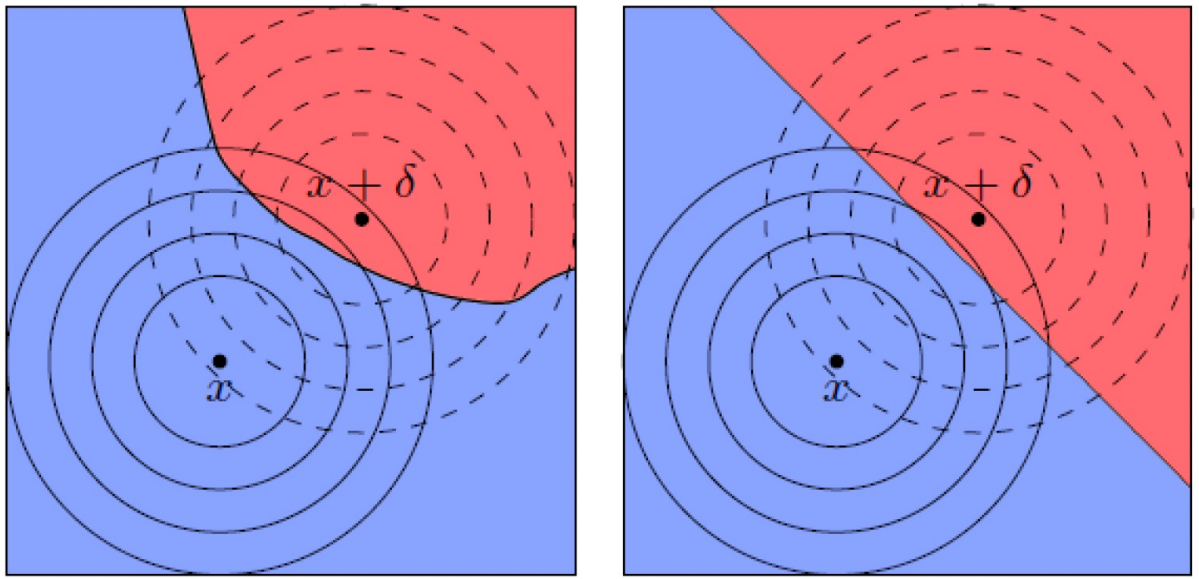
# Proof sketch (binary classifier)

1. Suppose the top class has probability $p_A$, so $f$ classifies $\mathcal{N}(x, \sigma^2 I)$ as $A$ with probability $\geq p_A$.
2. Consider a fixed perturbation $\delta$. We want the probability that $f$ classifies $\mathcal{N}(x + \delta, \sigma^2 I)$ as $A$. If this probability is greater than $1/2$ then $g(x + \delta) = A$.
3. We want a statement for all possible $f$, so consider the worst case $f$ which classifies $\mathcal{N}(x, \sigma^2 I)$ with probability $\geq p_A$, but minimizes the probability that $\mathcal{N}(x + \delta, \sigma^2 I)$ is $A$.

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

WILL NOT BE INCLUDED IN THE EXAM

# Illustrating worst-case classifier in the proof




Among all classifiers f for which g(x) is blue with probability greater than a given threshold, and g(x+δ) is blue with minimal probability, the “worst-case” is linear classifier normal to direction of δ from x

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

WILL NOT BE INCLUDED IN THE EXAM

# Proof sketch

1. Suppose the top class has probability $p_A$, so $f$ classifies $\mathcal{N}(x, \sigma^2 I)$ as $A$ with probability $\geq p_A$.
.
2. Consider a fixed perturbation $\delta$. We want the probability that $f$ classifies s $\mathcal{N}(x+\delta, \sigma^2 I)$ as $A$. If this probability is greater than $1/2$ then $g(x+\delta) = A$.

3. We want a statement for all possible $f$, so consider the worst case $f$ which classifies s $\mathcal{N}(x, \sigma^2 I)$ with probability $\geq p_A$, but minimizes the probability that $\mathcal{N}(x+\delta, \sigma^2 I)$ is $A$.

4. By a similar argument to the Neyman Pearson lemma, this worst-case classifier is the linear classifier $f(x') = \begin{cases} A \text{ if } \delta^T(x' - x) \leq \sigma \|\delta\|_2 \Phi^{-1}(p_A) \\ B \text{ otherwise} \end{cases}$

5. For this worst case classifier, $f$ classifies $\mathcal{N}(x+\delta, \sigma^2 I)$ as $A$ with probability $\Phi\left(\Phi^{-1}(p_A) - \frac{\|\delta\|_2}{\sigma}\right)$. Solving this for $1/2$ we get the condition $\|\delta\|_2 < \sigma \Phi^{-1}(p_A)$.

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Randomized Smoothing Guarantee

**Certified robust radius by [Cohen et al.'19] (for binary classification)**

Confidence of majority vote

Given any input $x \in \mathbb{R}^d$, let $\eta$ be Gaussian noise $\mathcal{N}(0, \sigma^2 I)$ and $p = \max_y \mathbb{P}_\eta[f(x+\eta) = y]$ . Then $g(x) = g(x+\delta)$ for any $\delta$ such that $\|\delta\|_2 \leq \Phi^{-1}(p)\sigma$, where $\Phi$ is CDF of standard Gaussian.

Computable certified radius for *x*

In a multiclass setting:

- If we can estimate that the probability $g(x) = A$ is at least $p_1$ and the probability that $g(x) = B$ is at most $p_2$, where A is the most likely class and B is the "runner-up" class, then above bound holds with $\Phi^{-1}(p)$ replaced by $(\Phi^{-1}(p_1) - \Phi^{-1}(p_2))/2$

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

**Pseudocode** for certification and prediction

*# evaluate $g$ at $x$*
**function** PREDICT($f$, $\sigma$, $x$, $n$, $\alpha$)
  counts $\leftarrow$ SAMPLEUNDERNOISE($f$, $x$, $n$, $\sigma$)
  $\hat{c}_A, \hat{c}_B \leftarrow$ top two indices in counts
  $n_A, n_B \leftarrow$ counts$[\hat{c}_A]$, counts$[\hat{c}_B]$
  **if** BINOMPVALUE($n_A$, $n_A + n_B$, 0.5) $\leq \alpha$ **return** $\hat{c}_A$
  **else return** ABSTAIN

*# certify the robustness of $g$ around $x$*
**function** CERTIFY($f$, $\sigma$, $x$, $n_0$, $n$, $\alpha$)
  counts0 $\leftarrow$ SAMPLEUNDERNOISE($f, x, n_0, \sigma$)
  $\hat{c}_A \leftarrow$ top index in counts0
  counts $\leftarrow$ SAMPLEUNDERNOISE($f, x, n, \sigma$)
  $\underline{p_A} \leftarrow$ LOWERCONFBOUND(counts$[\hat{c}_A]$, $n$, $1 - \alpha$)
  **if** $\underline{p_A} > \frac{1}{2}$ **return** prediction $\hat{c}_A$ and radius $\sigma\, \Phi^{-1}(\underline{p_A})$
  **else return** ABSTAIN

**Proposition 1.** *With probability at least $1 - \alpha$ over the randomness in* PREDICT, PREDICT *will either abstain or return $g(x)$. (Equivalently: the probability that* PREDICT *returns a class other than $g(x)$ is at most $\alpha$.)*

The function SAMPLEUNDERNOISE($f$, $x$, num, $\sigma$) in the pseudocode draws num samples of noise, $\varepsilon_1 \ldots \varepsilon_{\text{num}} \sim \mathcal{N}(0, \sigma^2 I)$, runs each $x + \varepsilon_i$ through the base classifier $f$, and returns a vector of class counts. BINOMPVALUE($n_A$, $n_A + n_B$, $p$) returns the p-value of the two-sided hypothesis test that $n_A \sim \text{Binomial}(n_A + n_B, p)$.



Cohen et al., “Certified Adversarial Robustness via Randomized Smoothing”, 2019.

# Binomial Hypothesis Testing

**if** BINOMPVALUE$(n_A, n_A + n_B, 0.5) \leq \alpha$ **return** $\hat{c}_A$
**else return** ABSTAIN

- Randomized smoothing uses $n$ noisy samples to approximate the expectation over an infinite number of draws.

- To ensure that $\hat{c}_A$ is truly more likely than $\hat{c}_B$, we test the following hypothesis:
  - **Null Hypothesis ($H_0$):** $P(\text{class } \hat{c}_A \mid \hat{c}_A \text{ or } \hat{c}_B) \leq 0.5$

    *(Class $A$ is **no more** likely than class $B$; any lead $n_A > n_B$ was pure luck).*
  - **Alternative Hypothesis ($H_1$):** $P(\text{class } \hat{c}_A \mid \hat{c}_A \text{ or } \hat{c}_B) > 0.5$

    *(Class $A$ is strictly more likely than class $B$).*

# Binomial Hypothesis Testing

**if** BINOMPVALUE$(n_A, n_A + n_B, 0.5) \leq \alpha$ **return** $\hat{c}_A$
**else return** ABSTAIN

- BINOMPVALUE$(n_A, n_A + n_B, 0.5)$
  - Flip a fair coin ($p = 0.5$) $n_A + n_B$ times
  - The probability of obtaining $\boldsymbol{n_A}$ **or more** "heads" (votes for $A$) purely by chance under the null hypothesis:

$$\text{p-value} = \sum_{k=n_A}^{n_A+n_B} \binom{n_A + n_B}{k} (0.5)^{n_A+n_B}$$

Recall: Prob of getting $k$ successes in $n$ trials:

$$\Pr(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

- **If** p-value $\leq \alpha$**:** We reject $H_0$. We are at least $(1 - \alpha)$ confident that $g(x) = \hat{c}_A$, allowing the function to safely return $\hat{c}_A$.
- **If** p-value $> \alpha$**:** The sample size $n$ is too small or the margin between $n_A$ and $n_B$ is too narrow to rule out a tie or mistake. Rather than making an uncertified or potentially wrong prediction, the function returns `ABSTAIN`.

*# certify the robustness of g around x*

**function** CERTIFY($f, \sigma, x, n_0, n, \alpha$)

counts0 $\leftarrow$ SAMPLEUNDERNOISE($f, x, n_0, \sigma$)

$\hat{c}_A \leftarrow$ top index in counts0

counts $\leftarrow$ SAMPLEUNDERNOISE($f, x, n, \sigma$)

$\underline{p_A} \leftarrow$ LOWERCONFBOUND(counts$[\hat{c}_A], n, 1-\alpha$)

**if** $\underline{p_A} > \frac{1}{2}$ **return** prediction $\hat{c}_A$ and radius $\sigma\, \Phi^{-1}(\underline{p_A})$

**else return** ABSTAIN

Have an initial estimate on $\hat{c}_A$ with a small number of samples.

LowerConfBound(k, n, 1−α) returns one-sided (1-α) lower interval for the Binomial parameter p given the sample k ~ Binomial(n,p)



Cohen et al., “Certified Adversarial Robustness via Randomized Smoothing”, 2019.

# Implementing Certified Robustness

*# certify the robustness of $g$ around $x$*
**function** CERTIFY($f$, $\sigma$, $x$, $n_0$, $n$, $\alpha$)
  counts0 $\leftarrow$ SAMPLEUNDERNOISE($f, x, n_0, \sigma$)
  $\hat{c}_A \leftarrow$ top index in counts0
  counts $\leftarrow$ SAMPLEUNDERNOISE($f, x, n, \sigma$)
  $\underline{p_A} \leftarrow$ LOWERCONFBOUND(counts$[\hat{c}_A]$, $n$, $1-\alpha$)
  **if** $\underline{p_A} > \frac{1}{2}$ **return** prediction $\hat{c}_A$ and radius $\sigma\, \Phi^{-1}(\underline{p_A})$
  **else return** ABSTAIN

---

**Proposition 2.** *With probability at least $1-\alpha$ over the randomness in* CERTIFY, *if* CERTIFY *returns a class $\hat{c}_A$ and a radius $R$ (i.e. does not abstain), then $g$ predicts $\hat{c}_A$ within radius $R$ around $x$: $g(x+\delta) = \hat{c}_A \;\; \forall \, \|\delta\|_2 < R$.*

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Certified Robustness via Randomized Smoothing

- First method to give mathematical guarantees of robustness
- Robustness radius R depends on noise parameter σ and separation between top two classes in prediction of x
- There is accuracy – robustness trade-off
- Follow-up work studies theoretical limits of robustness guarantees

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

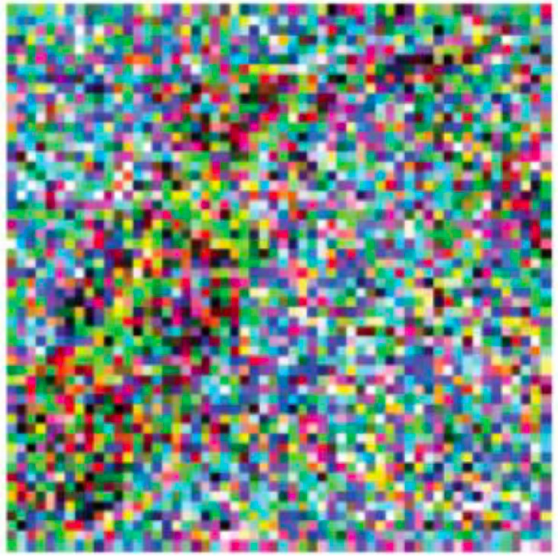
# Noise vs Resolution



Clean 56×56 image



Clean 224×224 image



Noisy 56×56 image
($\sigma = 0.5$)



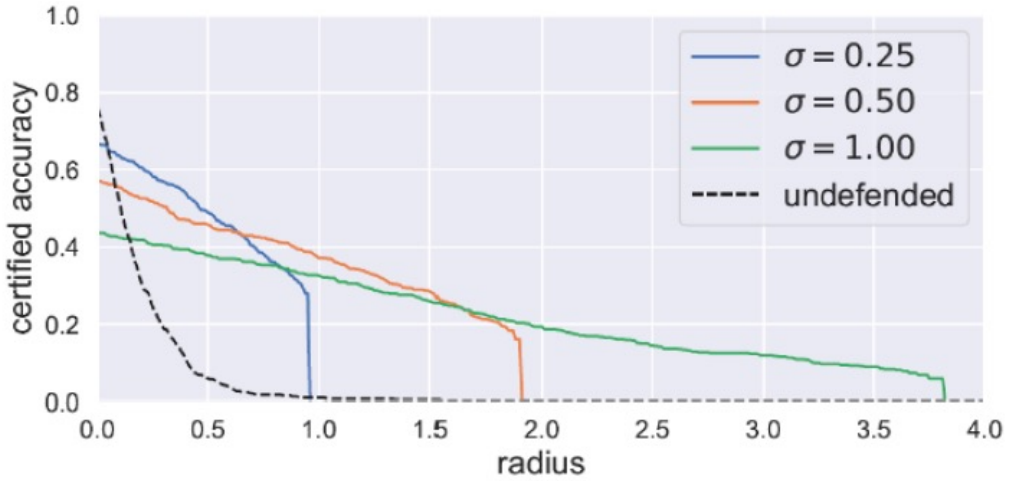
Noisy 224 ×224 image
($\sigma = 0.5$)

Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Certified Robustness: Empirical Evaluation

Plot of the certified top-1 accuracy by ResNet50 on ImageNet by the randomized smoothing

- As the radius *R* increases, the certified accuracy decreases
- The noise level $\sigma$ controls the tradeoff between accuracy and robustness
  - When $\sigma$ is small (e.g., $\sigma = 0.25$), perturbations with small radius *R* (e.g. $R = 0.5$) can be certified with high accuracy
  - However, for small $\sigma$ (e.g., $\sigma = 0.25$), perturbations with $R > 1.0$ cannot be certified
  - Increasing $\sigma$ (e.g., $\sigma = 1.0$) will enable robustness to larger perturbations ($R > 3.0$ and higher), but will result in decreased certified accuracy




Slide: Uni of Pennsylvania, Trustworthy ML - CIS 7000

# Randomized Smoothing During Training?

- The explained Randomized Smoothing is a post-processing/post-hoc method. The base classifier is not updated.
- Can we train/update the base classifier to have better certified robustness?
  - Noise Augmentation
  - Methods like MACER:

    Zhai et al., “MACER: Attack-free and Scalable Robust Training via Maximizing Certified Radius”, ICLR 2020.
- These can provide larger radius for certified robustness.